

Spatiotemporal AI

Tanujit Chakraborty



IEEE CIS Summer School 2026



“Space-time-causation, or name-and-form,
is what is called Maya.”

— Swami Vivekananda

Outline



1. Introduction: Time series, Spatial Time Series, and Forecasting
2. Graph-based representation
3. Graph Neural Networks and STGNNs₂
4. Handling Extremes with E-STGCN for AQI Forecasting
5. Open Data and Code
6. Conclusions and Future Directions

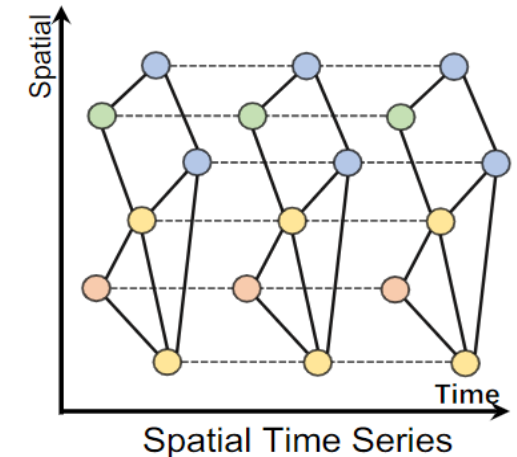
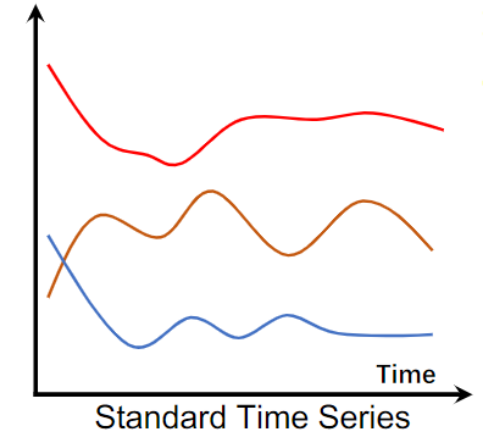


Introduction

Time and Space



- **Time series** is a set of observations; each one being recorded at a specific time.
- **Forecasting** is estimating how the sequence of observations will continue into the future.
- **Spatio-temporal data** is a type of data that combines spatial and temporal dimensions, capturing how measurements or observations vary across different locations over time.
- Spatiotemporal data can be modeled as **graph-structured data**, particularly when the data involves complex relationships, such as sensor networks, traffic flow, or social interactions.



Examples



Traffic monitoring



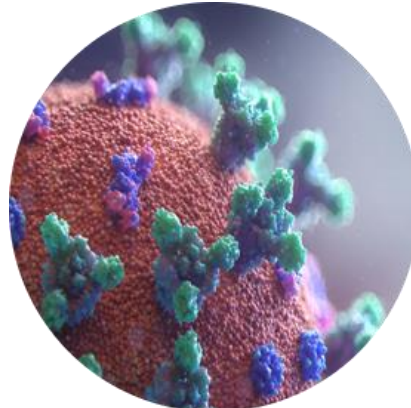
Weather monitoring



Smart cities



Social Interactions



Epidemics

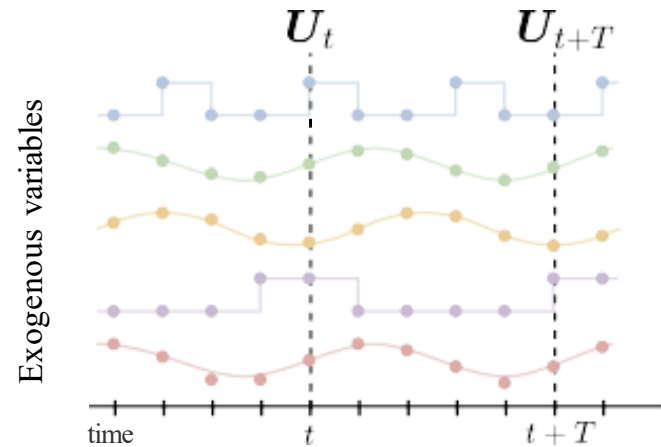
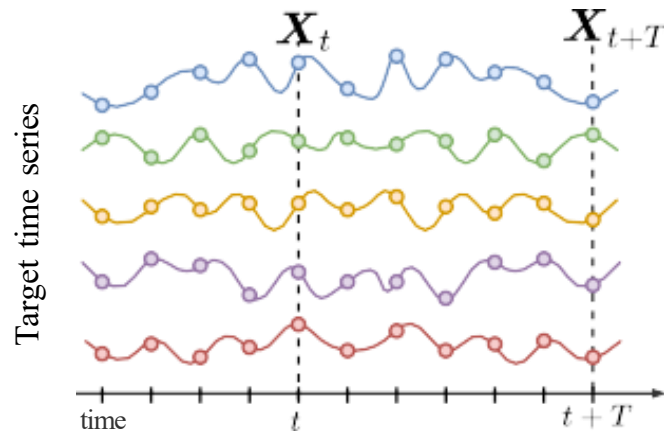


Stock markets

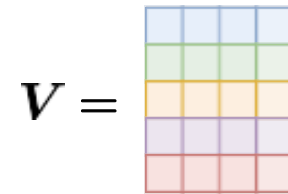
Collections of time series

We consider a set $\mathcal{D}$ of N **correlated time series**. Each i -th time series can be associated with:

- **observations** $\mathbf{x}_t^i \in \mathbb{R}^{d_x}$ at each time step t ;
- **exogenous variables** $\mathbf{u}_t^i \in \mathbb{R}^{d_u}$ at each time step t ;
- a vector of static (time-independent) **attributes** $\mathbf{v}^i \in \mathbb{R}^{d_v}$.



Static attributes



Capital letters denote the stacked N time series, i.e., $\mathbf{X}_t \in \mathbb{R}^{N \times d_x}$, $\mathbf{U}_t \in \mathbb{R}^{N \times d_u}$.

→ We call the dimension spanning the collection as **spatial** dimension.

Correlated time series

We consider a **time-invariant** stochastic process generating each time series as

$$\mathbf{x}_t^i \sim p^i(\mathbf{x}_t^i \mid \mathbf{X}_{<t}, \mathbf{U}_{\leq t}, \mathbf{V}) \text{ for all } i = 1 \dots N, t = 0, \dots, T - 1$$

and assume the existence of a **Granger causality** among time series.

Furthermore, time series are assumed to be

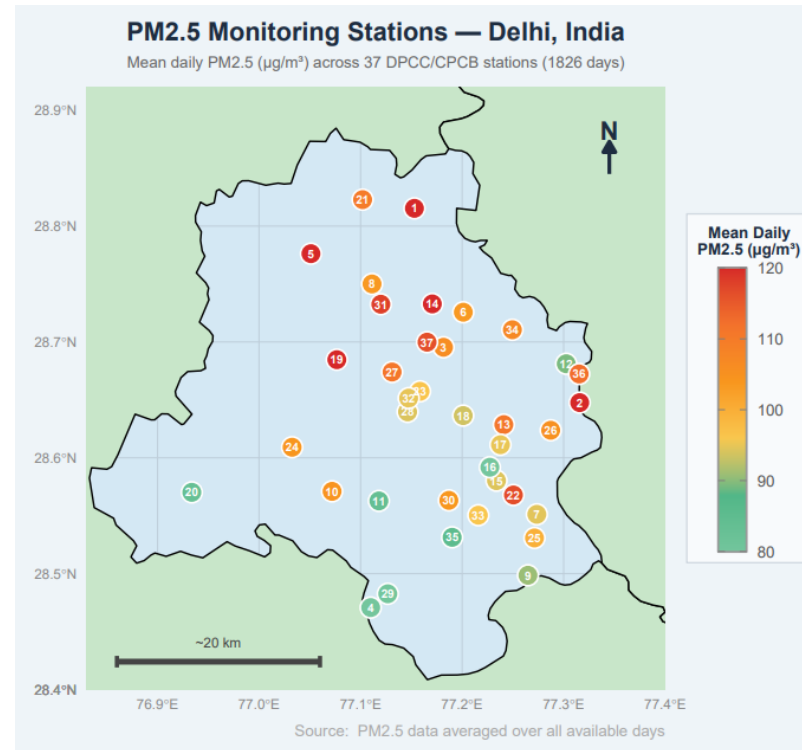
- a. homogenous,
- b. synchronous,
- c. regularly sampled.

- can be generated by **different processes**.

Notation: $\mathcal{X}_t = \langle \mathbf{X}_t, \mathbf{U}_t, \mathbf{V} \rangle$
 $X_t = [X_0, \dots, X_{t-2}, X_{t-1}]$

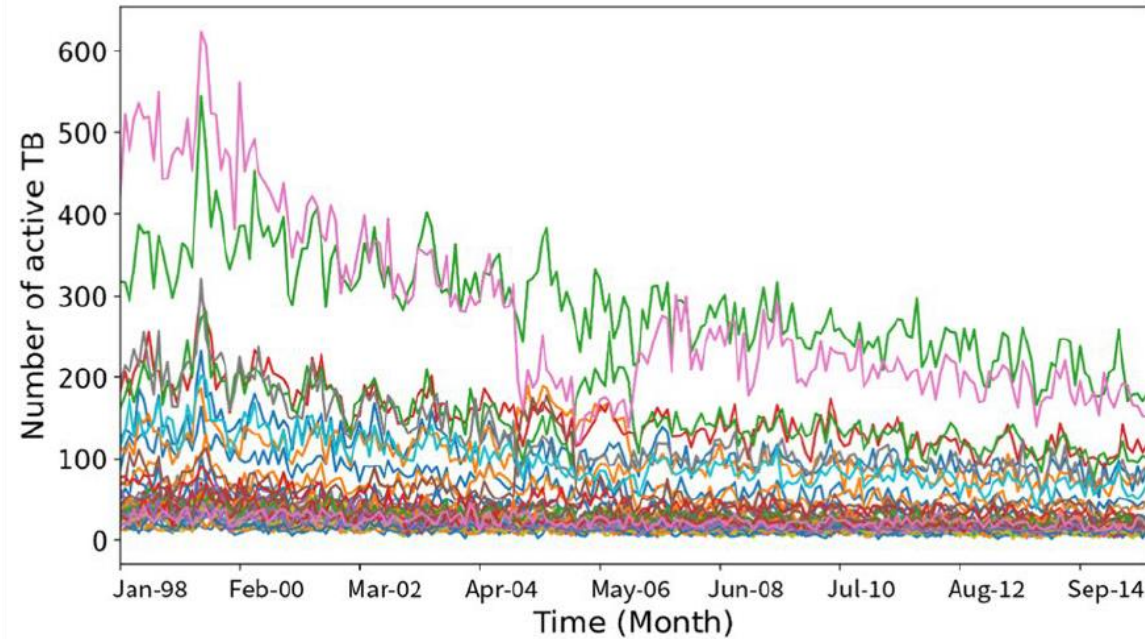
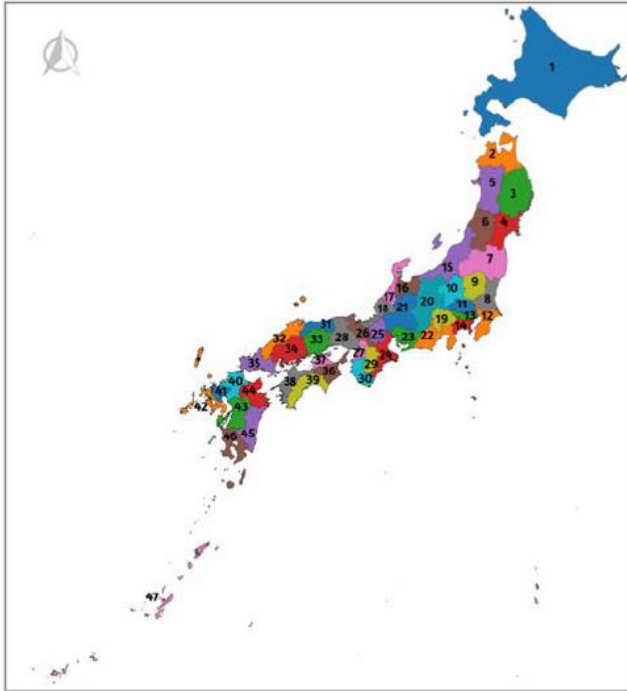
! Assumptions a, b, c can be relaxed (to be discussed later).

Example: Ground-station AQI monitoring system



- $X_{<t}$ collects ground-level measurements of several **air pollutant** variable.
 - U_t stores identifiers for **time-of-the-day** and **day-of-the-year**.
 - V collects static sensor's features, such as elevation, slope, and aspect of the station location.
- Time series **correlations** reflect the spatial proximity and topographic characteristics network.

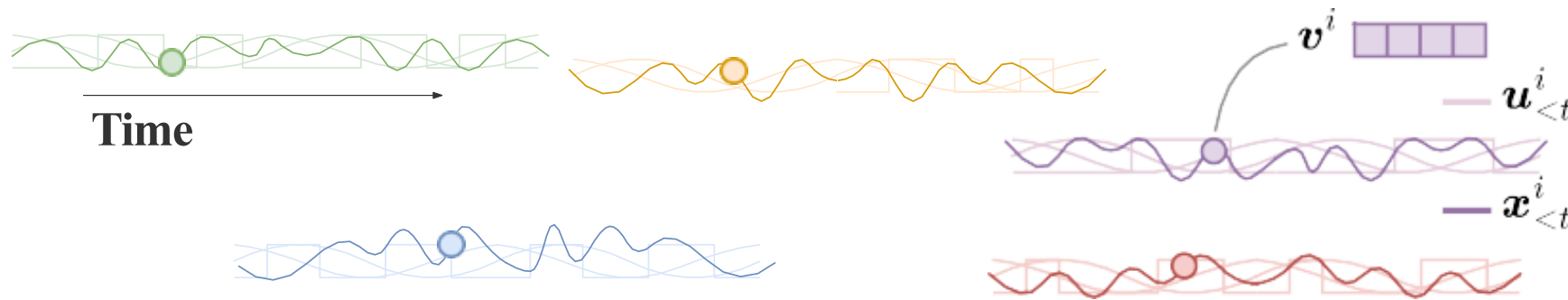
Example: Epidemics - Tuberculosis cases in Japan



- $X_{<t}$ collects monthly active TB cases recorded in each of Japan's 47 prefectures.
- U_t stores identifiers for month-of-the-year.
- V collects spatial locations (e.g., prefectures or provinces).

Example: Traffic monitoring system

Consider a sensor network monitoring the speed of vehicles or traffic flow at crossroads.



- $X_{<t}$ collects past **traffic** speed (or flow) **measurements**.
 - U_t stores identifiers for **time-of-the-day** and **day-of-the-week**.
 - V collects static sensor's features, e.g., **type** or **number of lanes** of the monitored road.
- Strong **dependencies** among time series that reflect the road network.

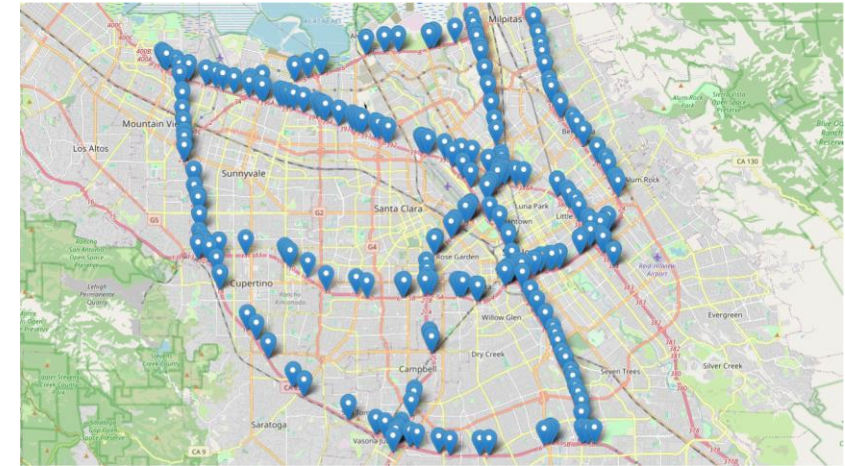
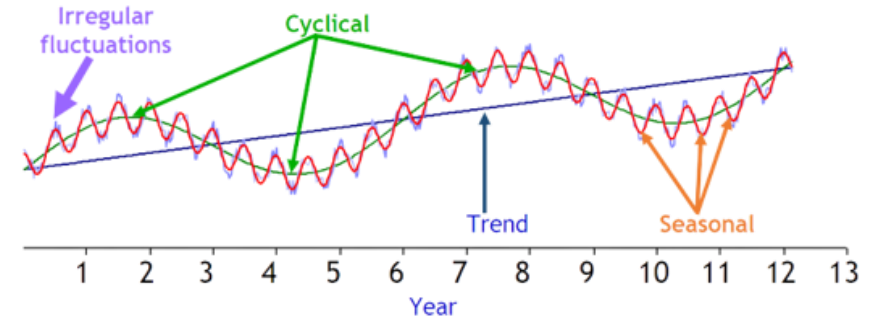
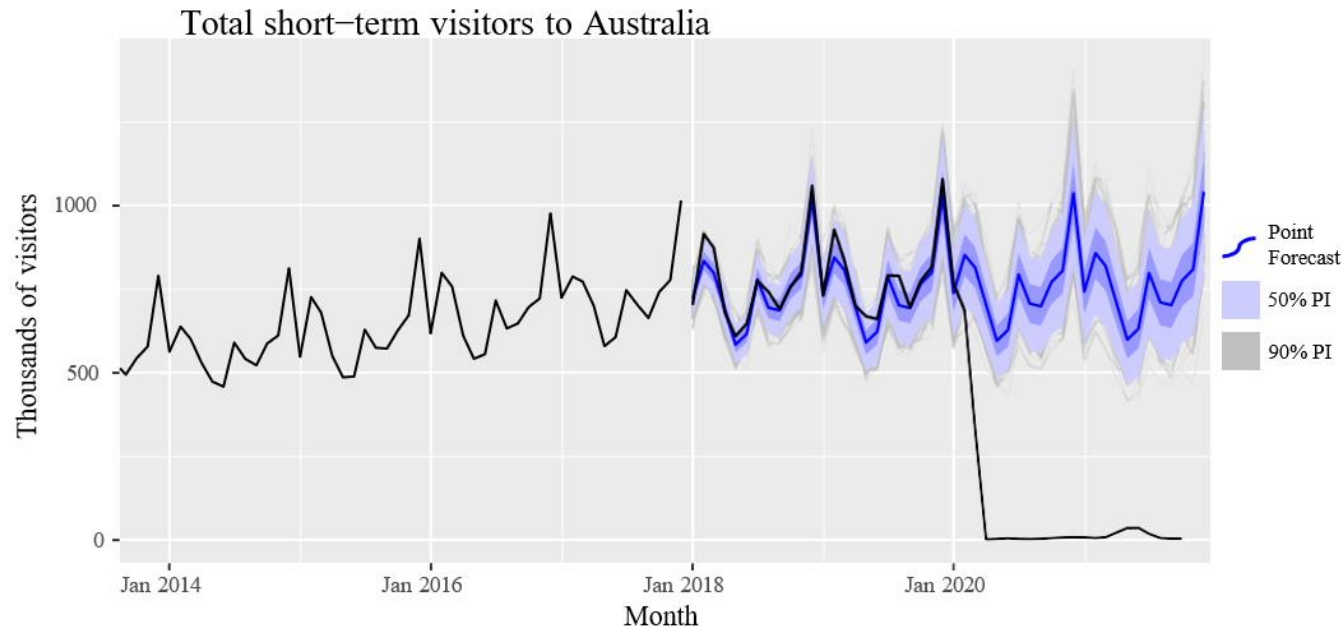


Fig: Sensor distribution of PeMS-BAY dataset

Time series forecasting



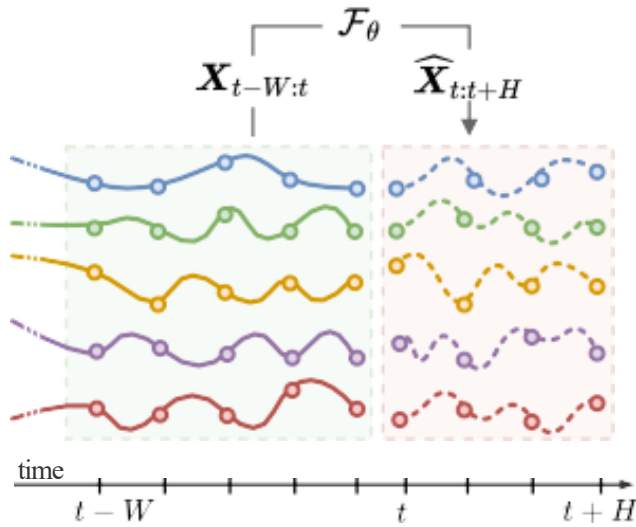
Forecasting is estimating how the sequence of observations will continue into the future.



Something is easy to forecast if:

1. we have a good **understanding of the factors** that contribute to it
2. there is a lot of **data available**
3. the future is somewhat **similar to the past**
4. forecasts can't affect the thing we try to forecast.

Forecasting



Multi-step time-series forecasting:

Given a window of $W \geq 1$ **past** values

$$\mathbf{X}_{t-W:t} = [\mathbf{X}_{t-W}, \dots, \mathbf{X}_{t-1}],$$

predict $H \geq 1$ **future** observations

$$\mathbf{X}_{t+h} \quad h = 1, \dots, H.$$

In particular, we are interested in **learning a model** $\mathcal{F}(\cdot, \cdot, \theta)$ s.t. $\mathcal{F}(\mathbf{X}_{t-W:t}, \mathbf{U}_{t:t+H}; \theta) = \hat{\mathbf{X}}_{t:t+H} \approx E_p[\mathbf{X}_{t:t+H}]$

Probabilistic predictors can be considered as well, but for now, we focus on point forecasts.

For point predictors, parameters θ can be learned by **minimizing a cost function** $\ell(\cdot, \cdot)$ (e.g., MSE) on a training set:

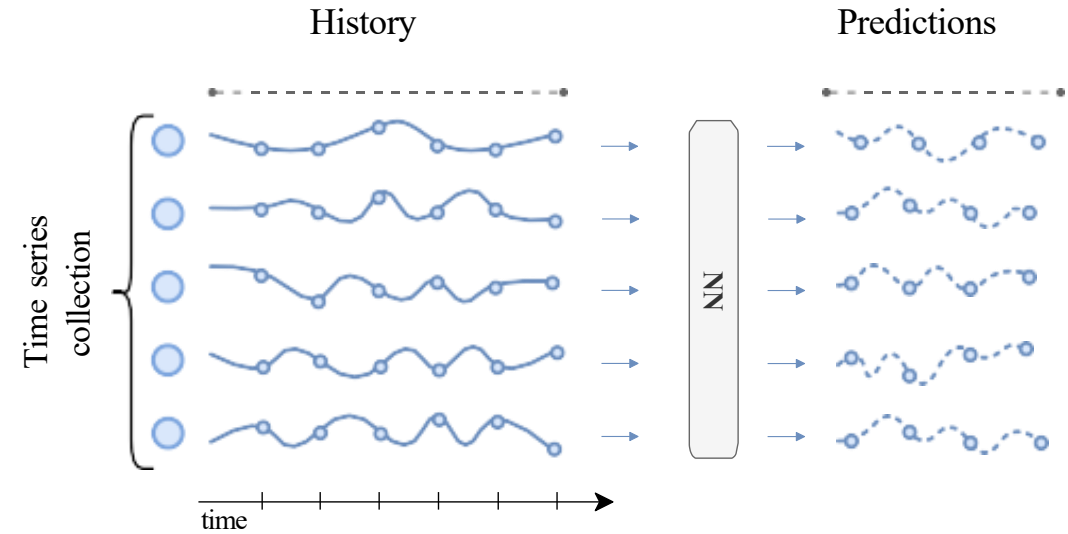
$$\hat{\theta} = \arg \min_{\theta} \frac{1}{NT} \sum_{t=1}^T \sum_{i=1}^N \ell \left(\hat{\mathbf{x}}_{t:t+H}^i, \mathbf{x}_{t:t+H}^i \right) = \arg \min_{\theta} \frac{1}{NT} \sum_{t=1}^T \sum_{i=1}^N \left\| \mathbf{x}_{t:t+H}^i - \hat{\mathbf{x}}_{t:t+H}^i \right\|_2^2.$$

Deep learning for time series



Modern deep learning methods for time series prediction tasks (e.g., forecasting) rely on a **single neural network** trained on a collection of **related time series**.

- ❑ Each time series is processed **independently**.
- ❑ Parameters are **shared**.
- ❑ Effective and **sample efficient**.
- ❑ **Spatial Dependencies are neglected**.

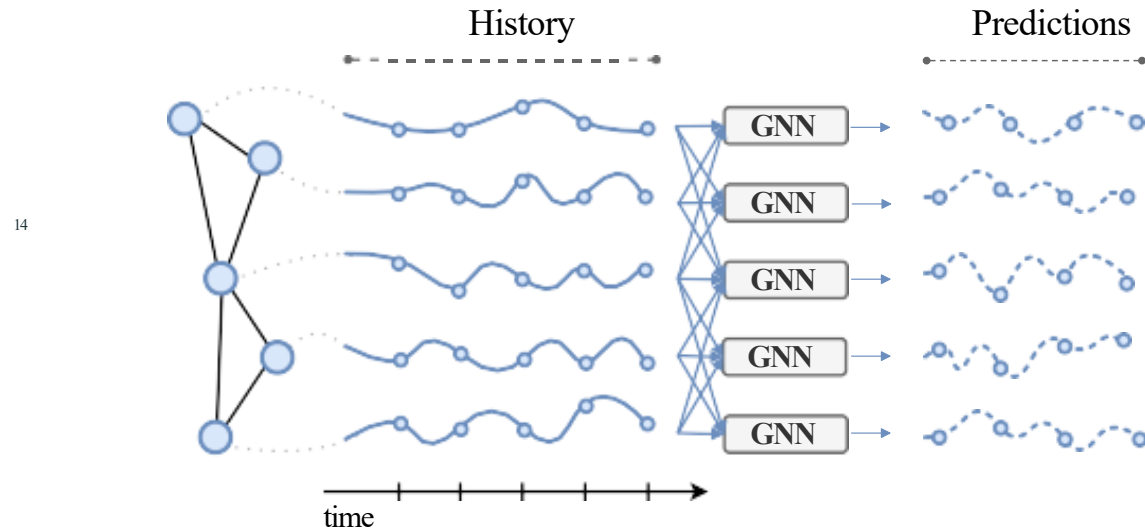


Graph deep learning for time series & spatiotemporal data



We will show graph deep learning (GDL) provides appropriate operators to go beyond these limitations.

- ❑ Dependencies are embedded into the processing as inductive biases.
- ❑ Operate on sets of correlated time series.
- ❑ Parameters are shared.



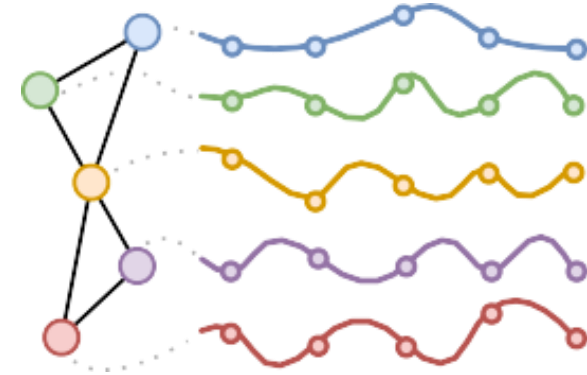
- ❑ There are inherent challenges in applying this processing to data from the real world.

What this tutorial is about



This tutorial presents advances coming from the combination of

1. deep learning for time series and
2. deep learning on graphs.



The objective of the tutorial is to provide:

1. a **comprehensive framework** for graph-based time series and spatiotemporal processing models;
2. methods to address challenges and potential pitfalls;
3. tools and guidelines for real-world applications and developing new methods.

This presentation is complemented by a Hands-on-coding session in Python.

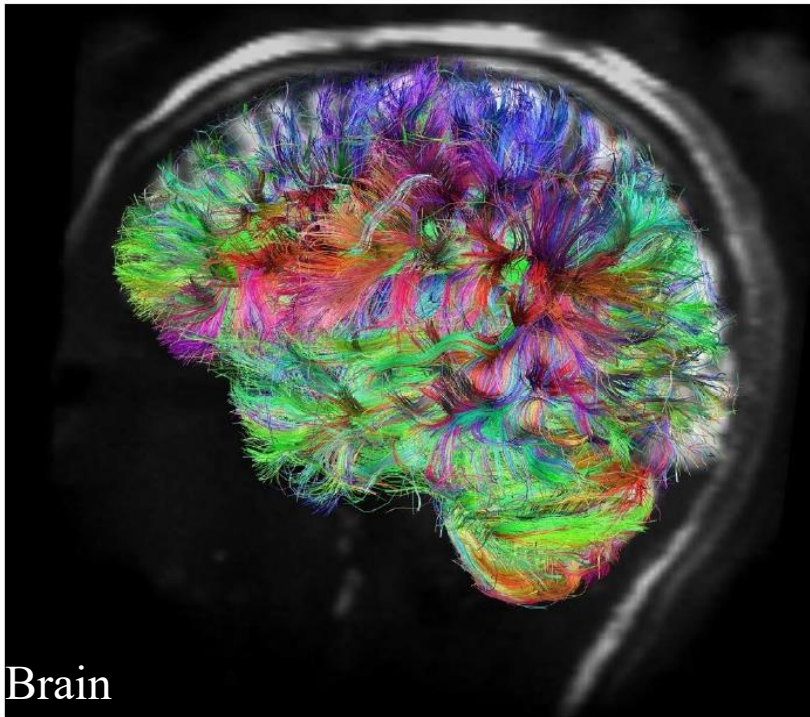
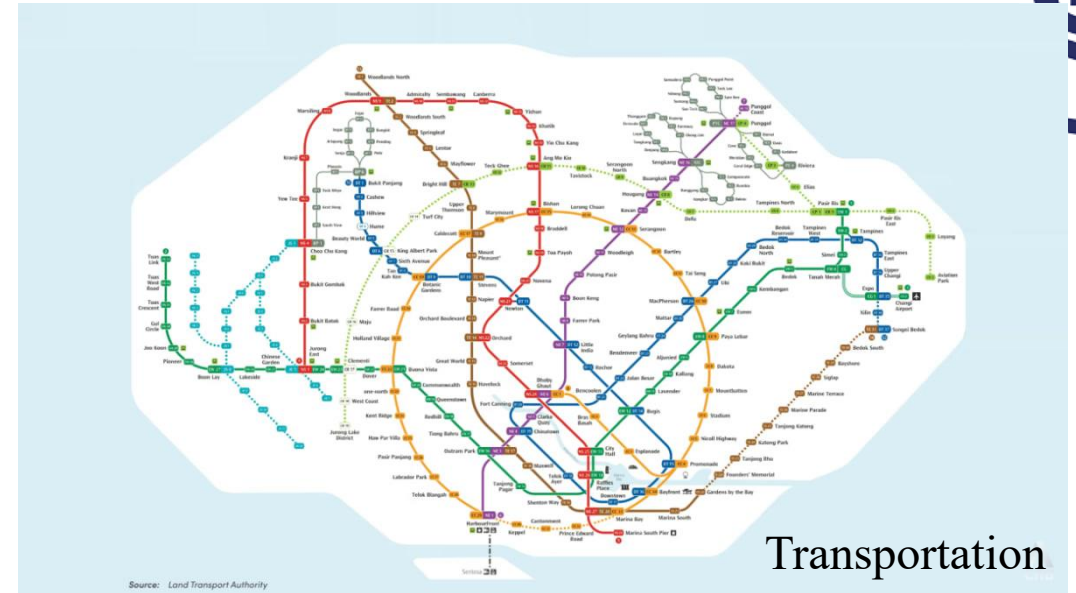


Graph-based representation

Graph-structured Data

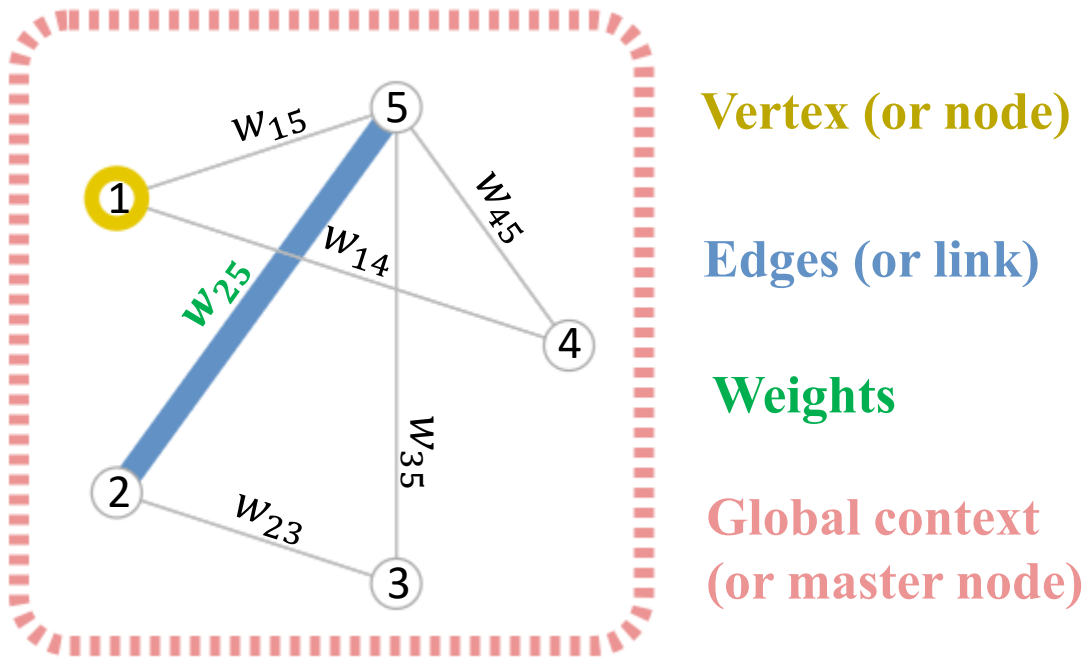
In many ways, **graphs** are the **main** modality of data we receive from **nature**.

Standard **CNN** and **RNN** architectures don't work on this data



What is a Graph?

- **Graph** represents the relations (edges) between a collection of entities (nodes) quantified using weights.
- **Graph $G(V, E, W)$** : V = nodes, E = edges (link node pairs), W = weights associated with *edges*.
- **Adjacency matrix (A)** of a graph is a sparse matrix with non-zero elements denoting graph weights.



$$A = \begin{bmatrix} 0 & 0 & 0 & w_{14} & w_{15} \\ 0 & 0 & w_{23} & 0 & w_{25} \\ 0 & w_{23} & 0 & 0 & w_{35} \\ w_{14} & 0 & 0 & 0 & w_{45} \\ w_{15} & w_{25} & w_{35} & w_{45} & 0 \end{bmatrix}$$

Adjacency matrix of a symmetric weighted graph

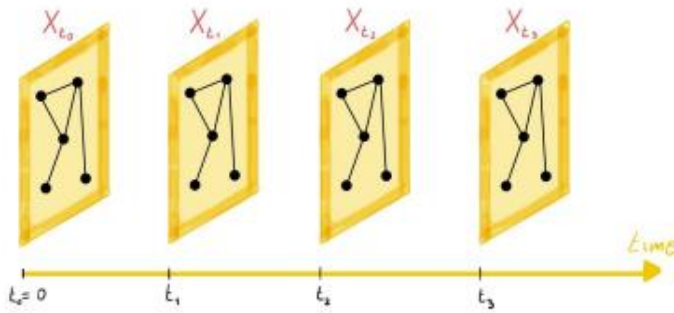
Fig: Attributes of undirected weighted graph

Dynamic Graphs



A dynamic graph is a type of graph whose structure (nodes and edges) and/or associated properties (attributes of nodes, X_t) change over time.

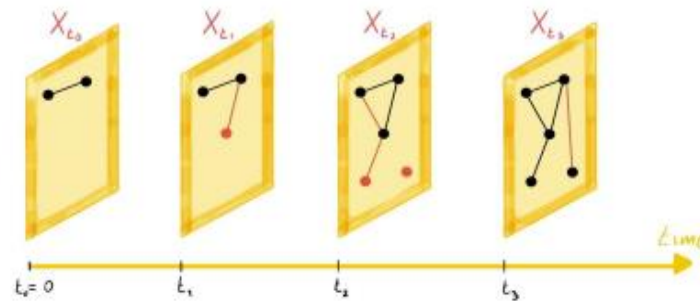
$$G_t = (V, E, X_t)$$



Spatio-Temporal Graph

- Topology is fixed, but features change over time
- (Usually) observed at regular intervals
- Examples: Air pollution level

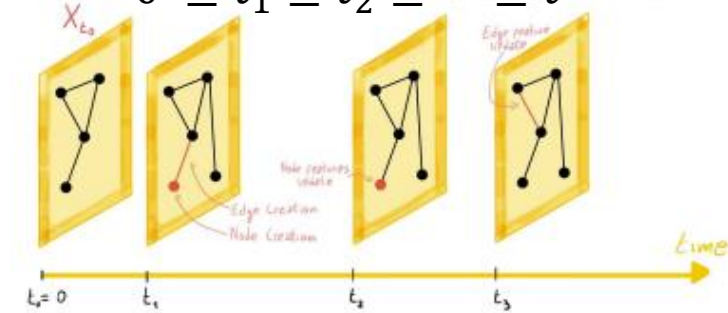
$$G_t = (V_t, E_t, X_t)$$



Discrete-Time Dynamic Graph

- Both topology and features change over time
- However, graph is observed at regular intervals
- Examples: Collaboration network

$$G(t) = \{x_{t_1}, x_{t_2}, \dots\}$$
$$0 \leq t_1 \leq t_2 \leq \dots \leq t$$



Continuous-Time Dynamic Graph

- Most general formulation
- Each change in the graph is observed individually with its timestamp
- Examples: Protein-protein interaction network

Relational information (with attributes)

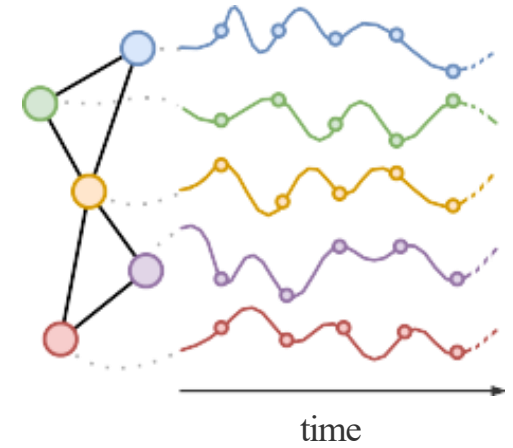
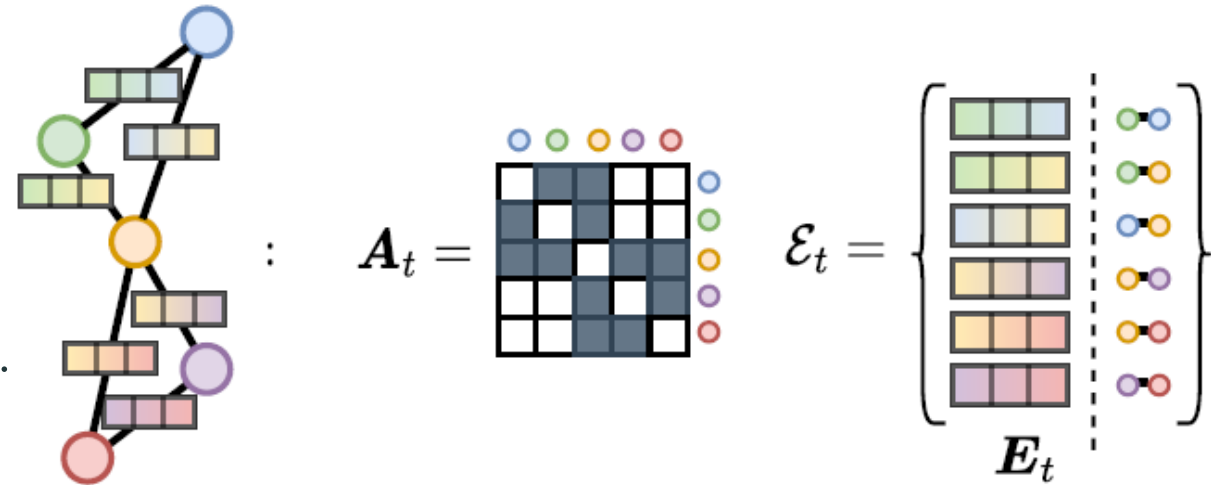


☁️ Exploit **functional dependencies** as an **inductive bias** to improve the forecasts.

- We can model pairwise relationships existing at time step t with **adjacency matrix** $A_t \in \{0, 1\}^{N \times N}$.
 - A_t can be **asymmetric** and **dynamic** (can vary with t).
- Optional **edge attributes** $e_t^{ij} \in \mathbb{R}^{d_e}$ can be associated to each non-zero entry of A_t .
- The set of attributed edges is denoted by

$$\mathcal{E}_t \doteq \left\{ \left\langle (i, j), e_t^{ij} \right\rangle \mid \forall i, j: A_t[i, j] \neq 0 \right\}.$$

→ Edge attributes can be both **categorical** or **numerical**.

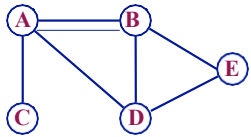


→ **Traffic monitoring system:** Edges in E can be obtained by considering the **road network**

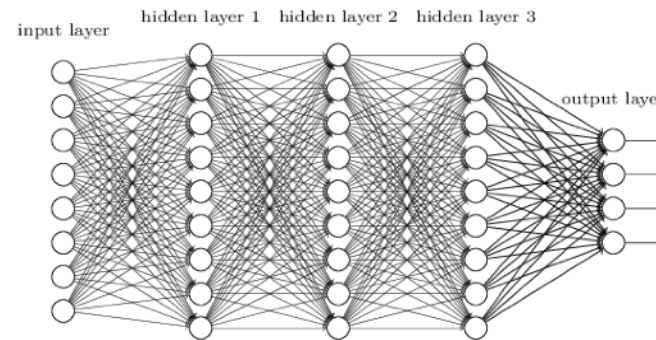
Road closures and traffic diversions can be accounted for with a dynamic topology $\mathcal{E}_t$

A naïve approach

- Take adjacency matrix: $\mathbf{A}$ and feature matrix: $\mathbf{X}$
- Concatenate them: $\mathbf{X}_{\text{in}} = [\mathbf{A}, \mathbf{X}]$ **Graph:** $G = (\mathcal{V}, \mathcal{E})$
- Feed them into deep (fully connected) neural net



	A	B	C	D	E	Feat	
A	0	1	1	1	0	1	0
B	1	0	0	1	1	0	0
C	1	0	0	1	0	0	1
D	1	1	1	0	1	1	1
E	0	1	0	1	0	1	0



? We need weight sharing!

Possible Solution:

Deep Learning on graphs or
Graph Neural (Convolutional) Networks

Problems:

- Huge number of parameters
- Re-train if graph changes

Model wish list:

- Trainable in $\mathcal{O}(|\mathcal{E}|)$ time
- Applicable even if the input graph changes

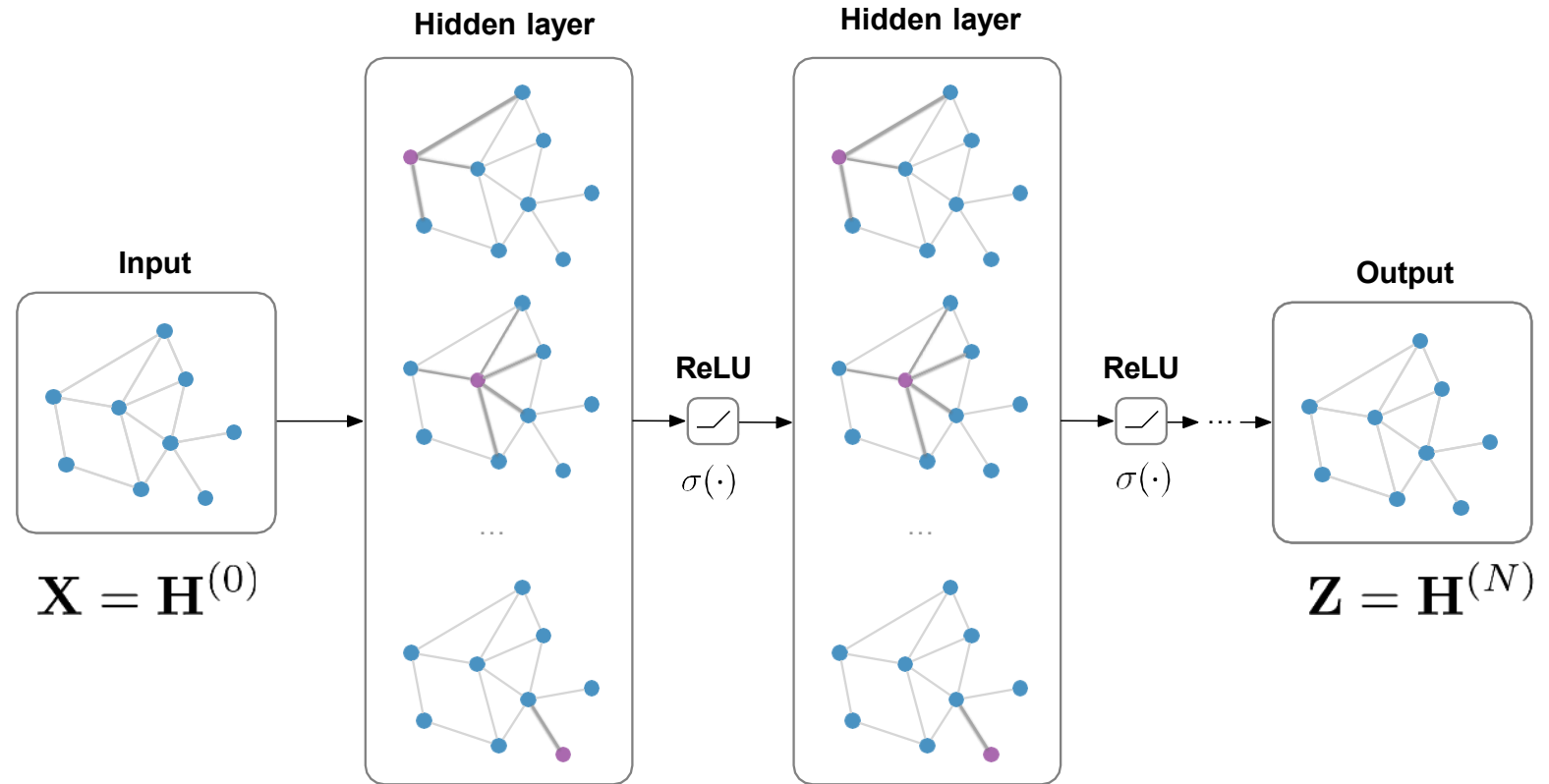


Graph Neural Networks

Graph Neural Networks (GNNs)

Notation: $\mathcal{G} = (\hat{\mathbf{A}}, \mathbf{X})$

- Adjacency matrix $\hat{\mathbf{A}} \in \mathbb{R}^{N \times N}$
- Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times E}$



$$\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{A}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right)$$

Main Idea (Gori et al. 2005): Pass messages between pairs of nodes and agglomerate

Alternative Interpretation: Pass messages between nodes to refine node (and possibly edge) representations

Neural message passing in GNNs

- GNNs utilize **neural message passing network** to exchange information amongst neighboring nodes and influence each other's updated embeddings as follows:
 - For each node in the graph, all the neighboring node embeddings (**messages**) are gathered.
 - All messages are aggregated via an **aggregate** function (like sum).
 - All pooled messages are passed through an **update function**, usually a learned neural network.

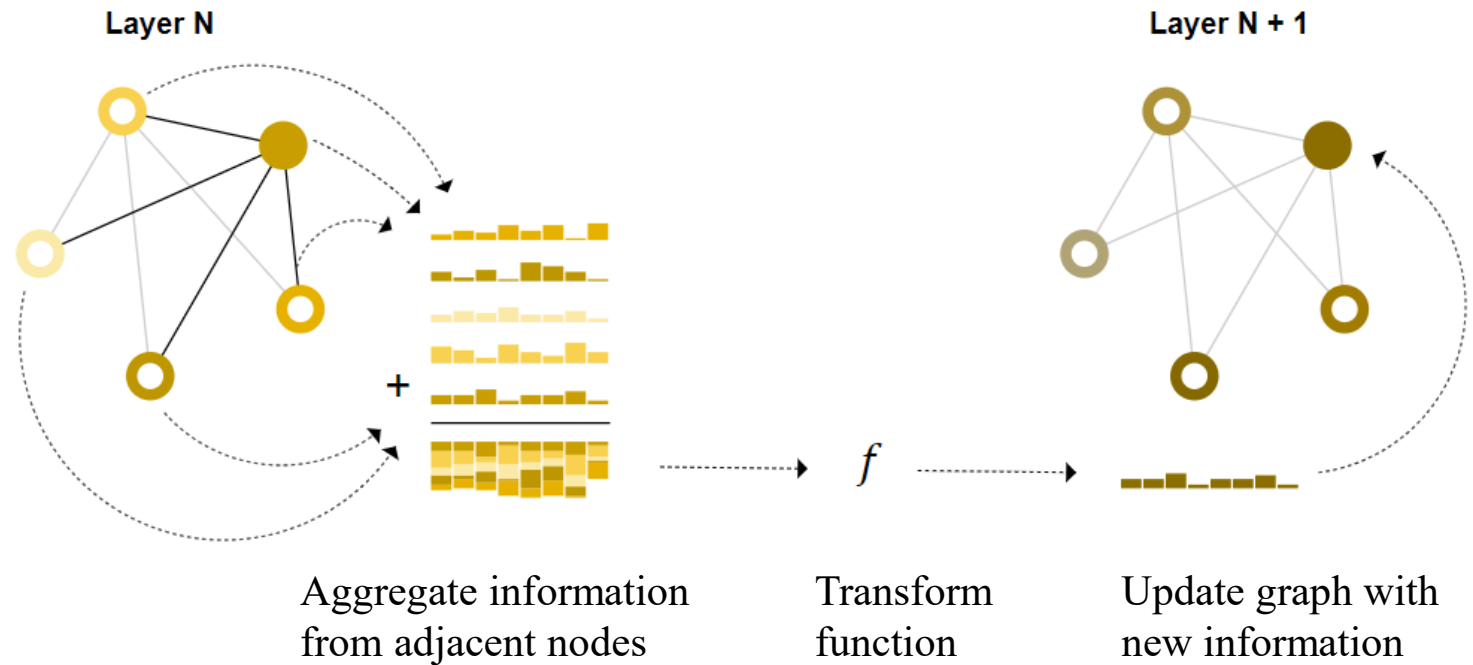
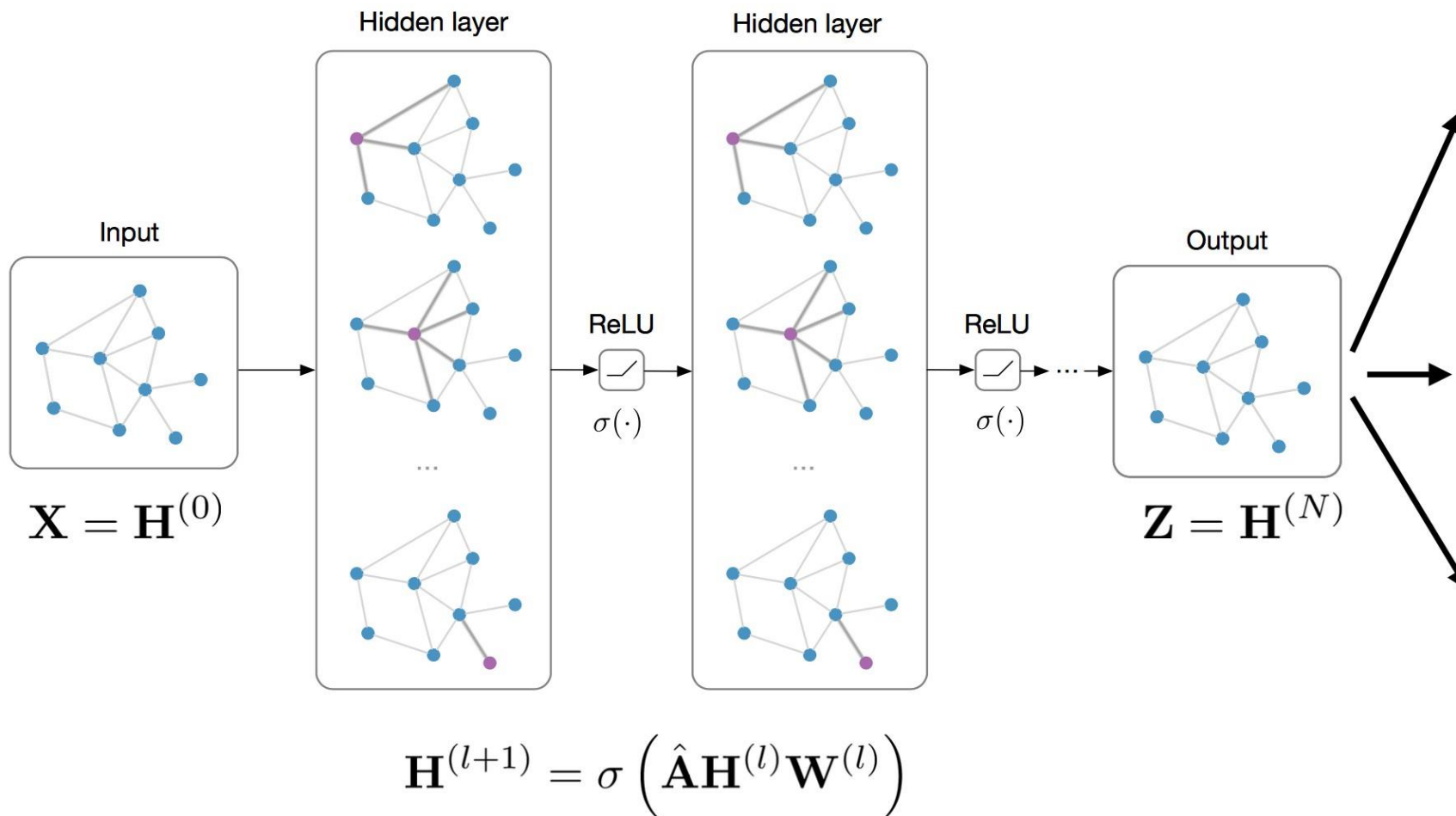


Fig: Updating the highlighted node embedding using adjacent nodes through neural message passing network.

Classification and Link Prediction with GNNs

Input: Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times E}$, preprocessed adjacency matrix $\hat{\mathbf{A}}$



Node classification:

$$\text{softmax}(\mathbf{z}_n)$$

e.g. Kipf & Welling (ICLR 2017)

Graph classification:

$$\text{softmax}(\sum_n \mathbf{z}_n)$$

e.g. Duvenaud et al. (NIPS 2015)

Link prediction:

$$p(A_{ij}) = \sigma(\mathbf{z}_i^T \mathbf{z}_j)$$

Kipf & Welling (NIPS BDL 2016)

“Graph Auto-Encoders”

Ref: Scarselli, Franco, et al. "The graph neural network model." IEEE transactions on neural networks 2008.

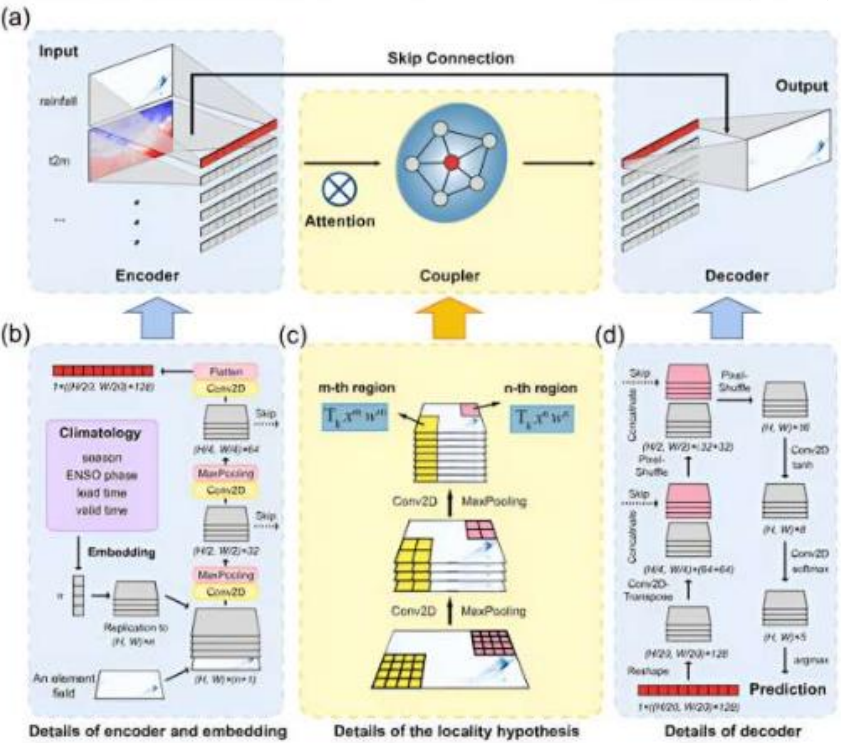
Kipf & Welling, “Semi-supervised classification with graph convolutional networks,” ICLR 2017.


 JANUARY 22, 2024

Novel graph neural network models enhance precipitation forecasting

by Chinese Academy of Sciences

 Geo. Res. Lett (2024)



VB
[The Machine](#)
[GamesBeat](#)
[Jobs](#)
[Special Issue](#)
[Become a Member](#)

The Machine
 Making sense of AI



DeepMind claims its AI improved Google Maps travel time estimates by up to 50%

Kyle Wiggers @Kyle_L_Wiggers September 3, 2020 7:00 AM




BBC
[Sign in](#)

[News](#)
[Sport](#)
[Reel](#)
[Worklife](#)
[Travel](#)
[Future](#)

NEWS

[Home](#)
[Video](#)
[World](#)
[UK](#)
[Business](#)
[Tech](#)
[Science](#)
[Stories](#)
[Entertainment & Arts](#)

BBC WORKLIFE

Our new guide for getting ahead

Food Discovery with Uber Eats: Using Graph Learning to Power Recommendations

Ankit Jain, Isaac Liu, Ankur Sarda, and Piero Molino

December 4, 2019

Scientists discover powerful antibiotic using AI


 21 February 2020
 [Share](#)

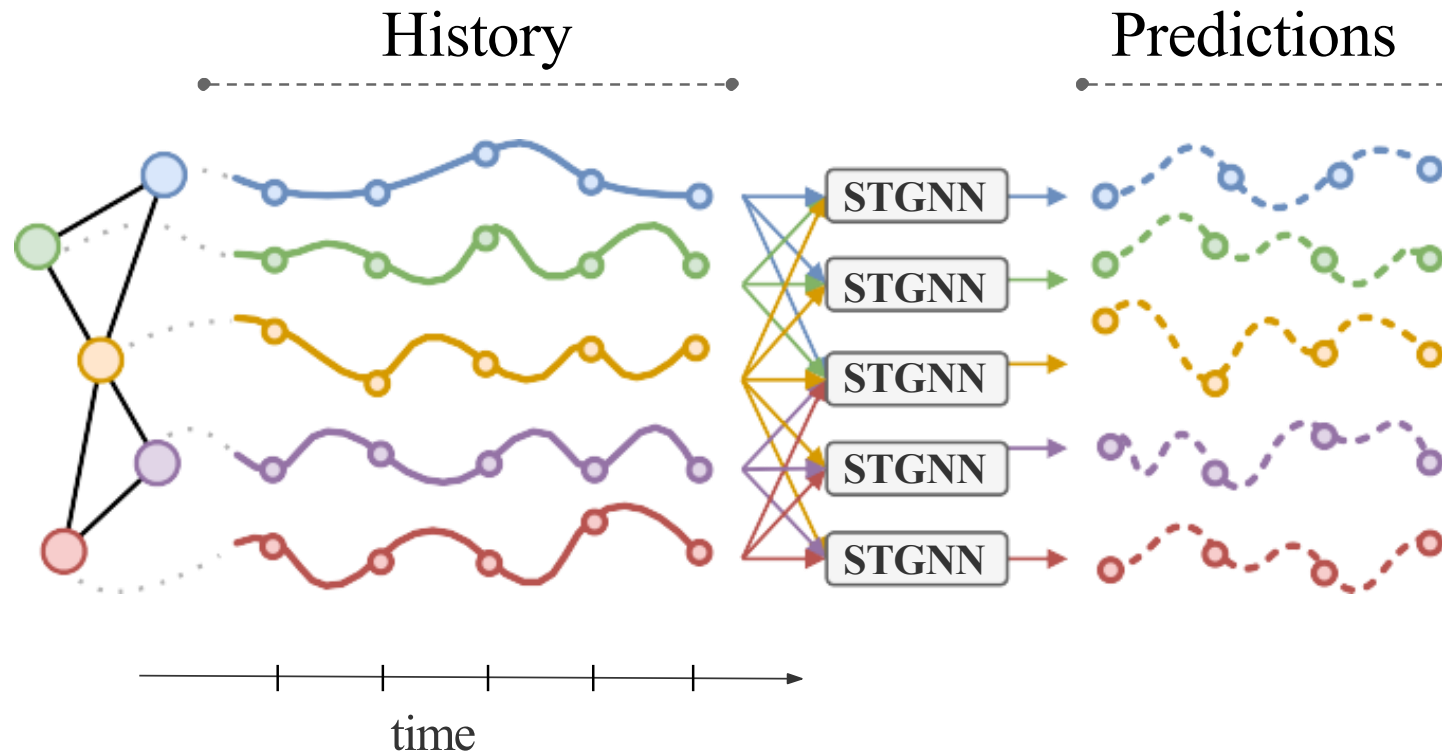




Spatiotemporal Graph Neural Networks

Spatiotemporal Graph Neural Networks

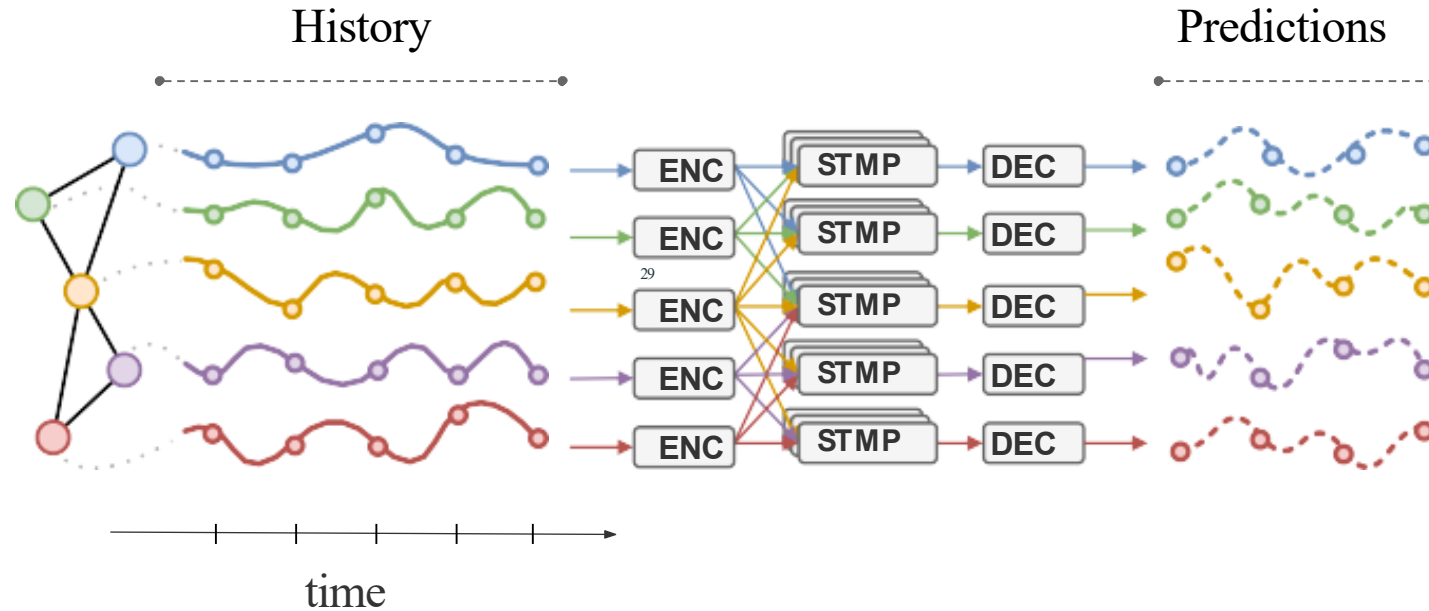
We call **spatiotemporal graph neural networks (STGNNs)** a neural network exploiting both temporal and spatial relations of the input spatiotemporal time series.



We focus on models based on **message passing (MP)**.

A general recipe for building STGNNs

We consider STGNNs consisting of three main components



- $\text{ENC}(\cdot)$ is the **encoding** layer, e.g., implemented by an MLP.
- $\text{STMP}(\cdot)$ is a stack of **spatiotemporal message-passing (STMP)** layers.
- $\text{DEC}(\cdot)$ is the **readout** layer, e.g., implemented by an MLP.

A closer look at STGNNs

Representations are updated as follows.

$$\begin{aligned} \mathbf{h}_{t-1}^{i,0} &= \text{ENCODER} \left(\mathbf{x}_{t-1}^i, \mathbf{u}_{t-1}^i, \mathbf{v}^i \right), \\ \mathbf{H}_{t-1}^{l+1} &= \text{STMP}^l \left(\mathbf{H}_{\leq t-1}^l, \mathcal{E}_{\leq t-1} \right), \quad l = 0, \dots, L-1 \\ \hat{\mathbf{x}}_{t:t+H}^i &= \text{DECODER} \left(\mathbf{h}_{t-1}^{i,L}, \mathbf{u}_{t:t+H}^i \right). \end{aligned}$$

- $\text{ENC}(\cdot)$ process each observation **independently**.
- $\text{STMP}(\cdot)$ is where **propagation** through time and space happens.
- $\text{DEC}(\cdot)$ maps each representation to **predictions**.

Spatiotemporal Message-Passing (STMP)

STMP blocks can be defined as:

$$\mathbf{h}_t^{i,l+1} = \text{UP}^l \left(\mathbf{h}_{\leq t}^{i,l}, \text{AGGR}_{j \in \mathcal{N}_t(i)} \left\{ \text{MSG}^l(\mathbf{h}_{\leq t}^{i,l}, \mathbf{h}_{\leq t}^{j,l}, \mathbf{e}_{\leq t}^{ji}) \right\} \right)$$

Each block processes **sequences** while accounting for **relational dependencies**.

As in standard MP operators:

- $\text{MSG}^l(\cdot)$ is a **message function**, e.g., implemented by *temporal convolutional layers*.
 - $\text{AGGR}\{\cdot\}$ is a permutation invariant **aggregation function**.
 - $\text{UP}^l(\cdot)$ is an **update function**, e.g., implemented by an RNN.
- ! Blocks can be implemented by composing MP and sequence modeling operators.
→ Many possible designs exist. Most popular is:
- ❖ **Time-and-Space (T&S):** Temporal and spatial processing cannot be factorized in two separate steps.

Time-and-Space (T&S)

In T&S models, representations at every node and time step are obtained by **jointly** propagating representation through time and space.

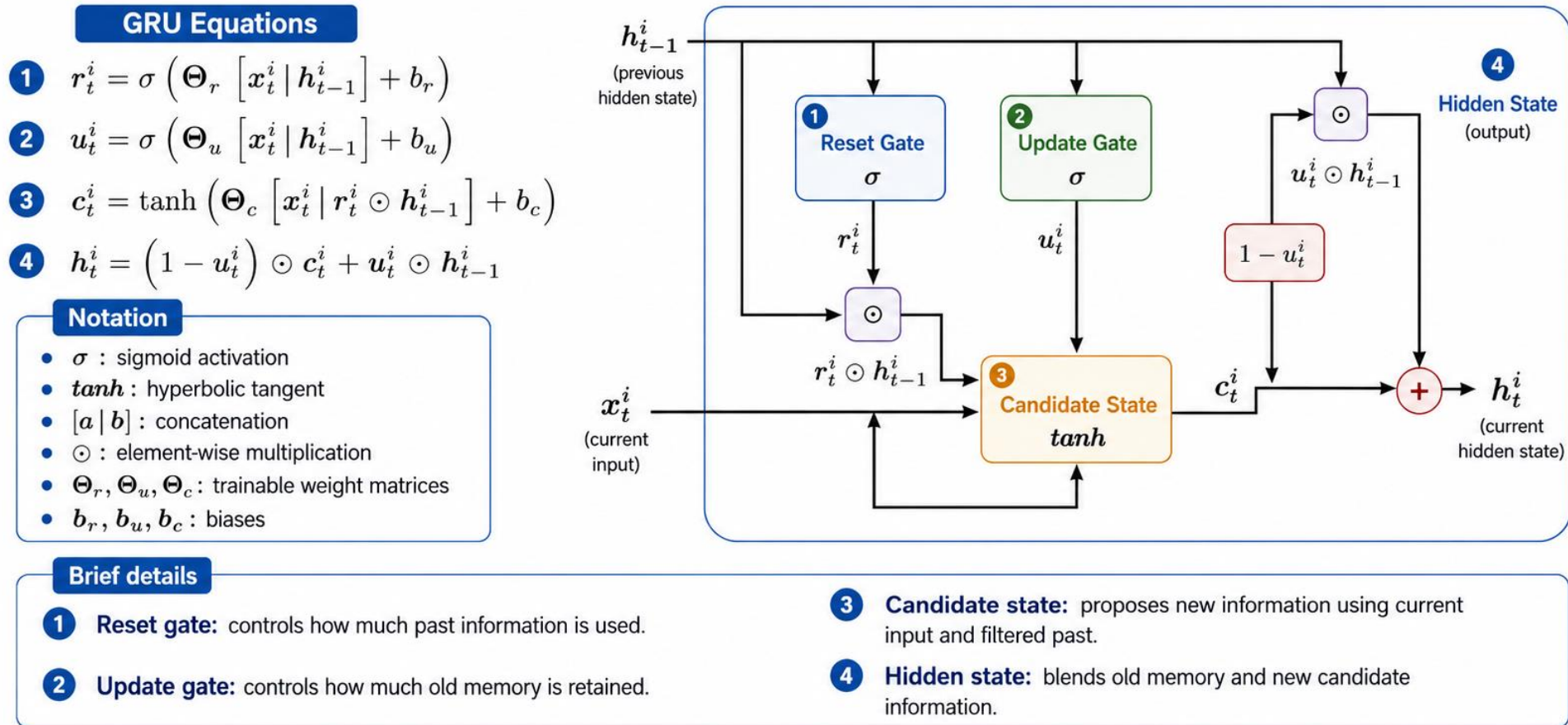
$$\mathbf{H}_{t-1}^{l+1} = \text{STMP}^l \left(\mathbf{H}_{\leq t-1}^l, \mathcal{E}_{\leq t-1} \right)$$

Several options exist:

- Integrate MP into neural operators for sequential data.
 - **Graph recurrent architectures, spatiotemporal convolutions, spatiotemporal attention, ...**
- Use sequence molding operators to compute messages.
 - **Temporal graph convolutions, spatiotemporal cross-attention, ...**
- Product graph representations (omitted here – see [*])

Example 1: From Recurrent Neural Networks...

Consider a **standard GRU cell**

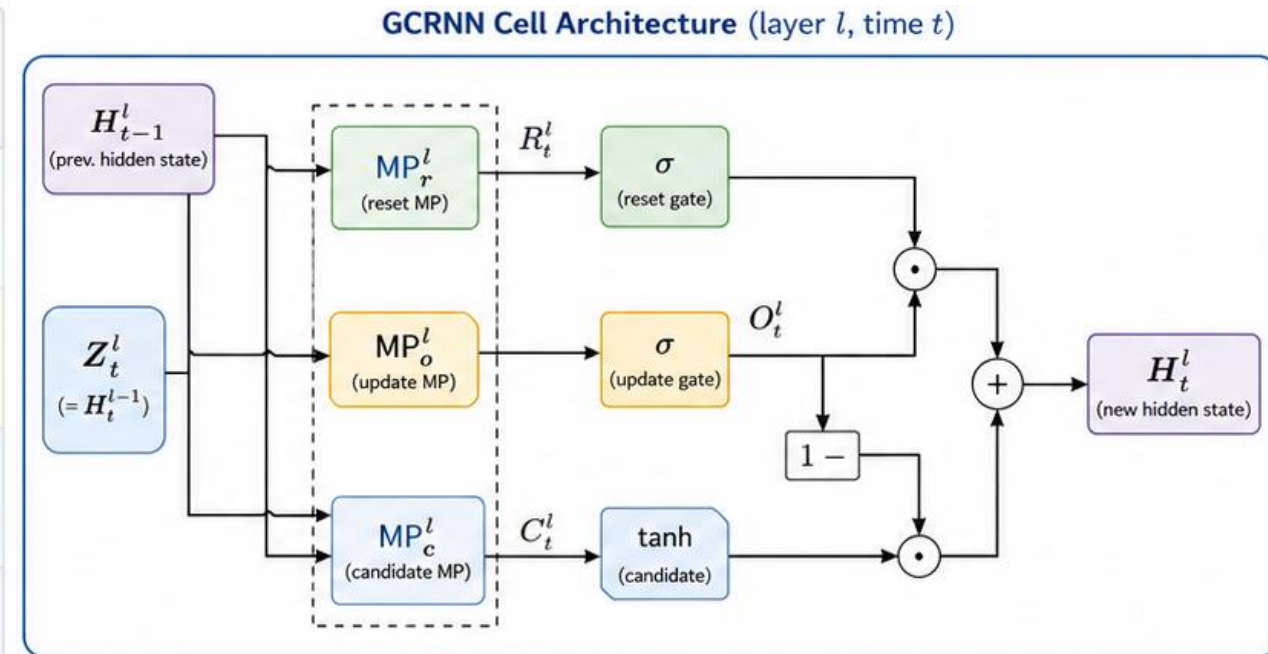


Time series are processed **independently** for each node or as a **single multivariate** time series.

...to Graph Convolutional RNNs

We can obtain a T&S model by implementing the gates of the GRU with MP blocks:

0	$Z_t^l = H_{t-1}^{l-1}$	The input to layer l at time t is the hidden state from the previous layer (usually $H_t^0 = X_t$, the node features).
1	$R_t^l = \sigma\left(\text{MP}_r^l\left([Z_t^l \parallel H_{t-1}^l], \mathcal{E}_t\right)\right)$	Reset gate. Controls how much of the previous hidden state to use when computing the candidate state.
2	$O_t^l = \sigma\left(\text{MP}_o^l\left([Z_t^l \parallel H_{t-1}^l], \mathcal{E}_t\right)\right)$	Update gate. Controls how much of the previous hidden state to keep in the current state.
3	$C_t^l = \tanh\left(\text{MP}_c^l\left([Z_t^l \parallel (R_t^l \odot H_{t-1}^l)], \mathcal{E}_t\right)\right)$	Candidate state. The reset gate filters the previous hidden state before message passing.
4	$H_t^l = O_t^l \odot H_{t-1}^l + (1 - O_t^l) \odot C_t^l$	New hidden state. A gated combination of the previous state and the candidate state.



- By replacing the linear transformations in a standard GRU with graph message passing operations, the model can capture both temporal dynamics and spatial dependencies on graphs.
- These T&S models are known as **graph convolutional recurrent neural networks** (GCRNNs).

Example 2: Spatiotemporal Convolutional Networks



Spatiotemporal convolutional networks (STCNs) instead **alternate spatial and temporal convolutions**:

1. Compute intermediate representations by using a **temporal convolutional** layer:

$$\mathbf{z}_{t-W:t}^{i,l} = \text{TCN}^l \left(\mathbf{h}_{t-W:t}^{i,l-1} \right) \quad \forall i$$

where TCN^l indicates a temporal convolutional layer.

2. Compute the updated representation at each time step by using a **graph convolution**:

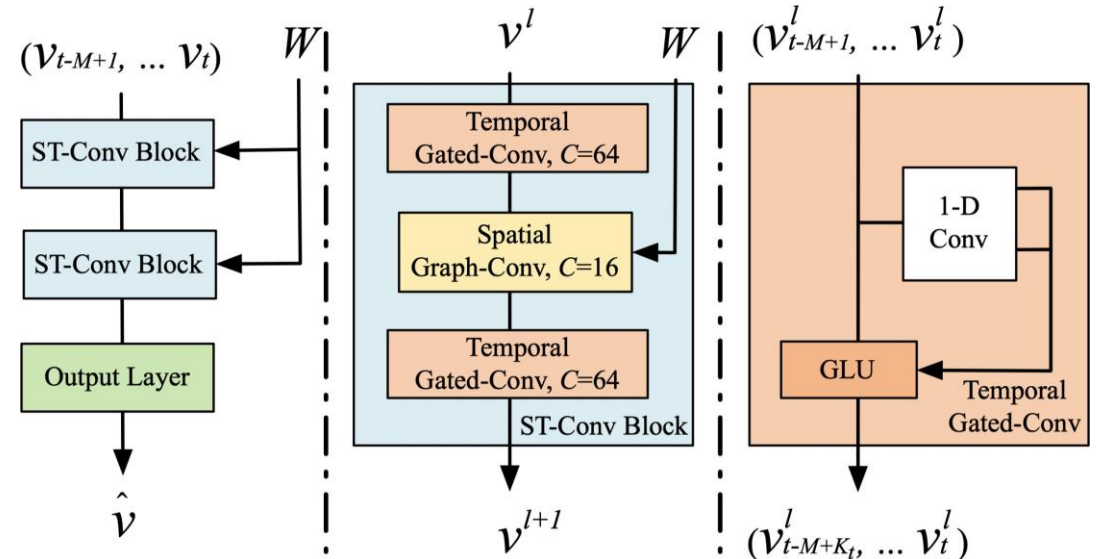
$$\mathbf{H}_t^l = \text{MP}^l \left(\mathbf{Z}_t^l, \mathcal{E}_t \right) \quad \forall t$$

Spatiotemporal Convolutional Networks

□ The first example of such architecture is the **STGCN** by Yu et al.

□ The model is obtained by stacking STMP blocks consisting of

- a (gated) temporal convolution;
- a polynomial graph convolution;
- a second (gated) temporal convolution.



Example 3: Temporal Graph Convolution

A more integrated approach instead consists of implementing a **temporal propagation** mechanism in the message function.

For example, we can design STMP layers s.t.

$$\mathbf{h}_{t-W:t}^{i,l} = \text{TCN}_1^l \left(\mathbf{h}_{t-W:t}^{i,l-1}, \text{AGGR}_{j \in \mathcal{N}_t(i)} \left\{ \text{TCN}_2^l \left(\mathbf{h}_{t-W:t}^{i,l-1}, \mathbf{h}_{t-W:t}^{j,l-1}, \mathbf{e}_{t-W:t}^{ji} \right) \right\} \right)$$

Analogous models can be built with **any sequence modeling architecture**.

→ **Example:** many rely on attention-based operators (Ref).

Ref: Marisca *et al.*, “Learning to Reconstruct Missing Data from Spatiotemporal Graphs with Sparse Observations”, NeurIPS 2022.

Wu *et al.*, “TraverseNet: Unifying Space and Time in Message Passing for Traffic Forecasting”, TNNLS 2022.

Handling Extremes in STGCN

Journal of the Royal Statistical Society Series A:
Statistics in Society, 2026, **00**, 1–51
<https://doi.org/10.1093/jrssa/qnag010>



Original Article

E-STGCN: extreme spatio-temporal graph convolutional networks for air quality forecasting

Madhurima Panja^{1,†} , Tanujit Chakraborty^{1,2,†} , Anubhab Biswas³
and Soudeep Deb⁴ 

¹SAFIR, Sorbonne University Abu Dhabi, Al Reem Island, Abu Dhabi 38044, UAE

²Sorbonne Center for Artificial Intelligence, Sorbonne University, Paris 75005, France

³SUPSI, University of Applied Sciences and Arts of Southern Switzerland, Manno, Switzerland

⁴Decision Sciences Area, Indian Institute of Management Bangalore, Bangalore, India

Address for correspondence: Tanujit Chakraborty. Email: tanujit.chakraborty@sorbonne.ae

Air Quality: A Global Concern

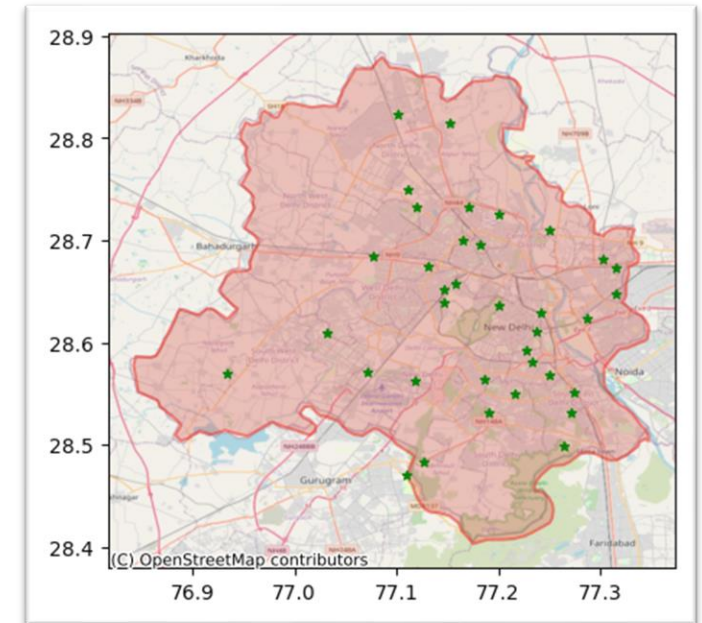


- Emission of particulate matter and harmful gases in the air from industrialization degrades air quality levels leading to harmful health hazards.
- WHO has set National Ambient Air Quality Standards (NAAQS) for major air pollutants to protect the public health and environment.
- Despite several efforts, Delhi (Capital of India) remains the most polluted capital city in the world as of early 2024.

Air Quality Data of Delhi



- In efforts to monitor and reduce the air pollution levels, the Central Pollution Control Board (CPCB) in Delhi has installed a network of forty ground-based sensors at different strategic locations.
- The sensors collect near real-time (hourly) data on the concentration levels of major air pollutants, e.g., PM_{10} , $\text{PM}_{2.5}$, NO_2 .
- This timely information empowers the pollution control management system, facilitating informed decision-making to reduce pollution levels.



Delhi Air Quality Data Characteristics



Daily pollutant concentration levels of $\text{PM}_{2.5}$, PM_{10} , NO_2 were obtained from 40 ground-based sensors of Delhi's CPCB during 1st January 2019 till 31st December 2023.

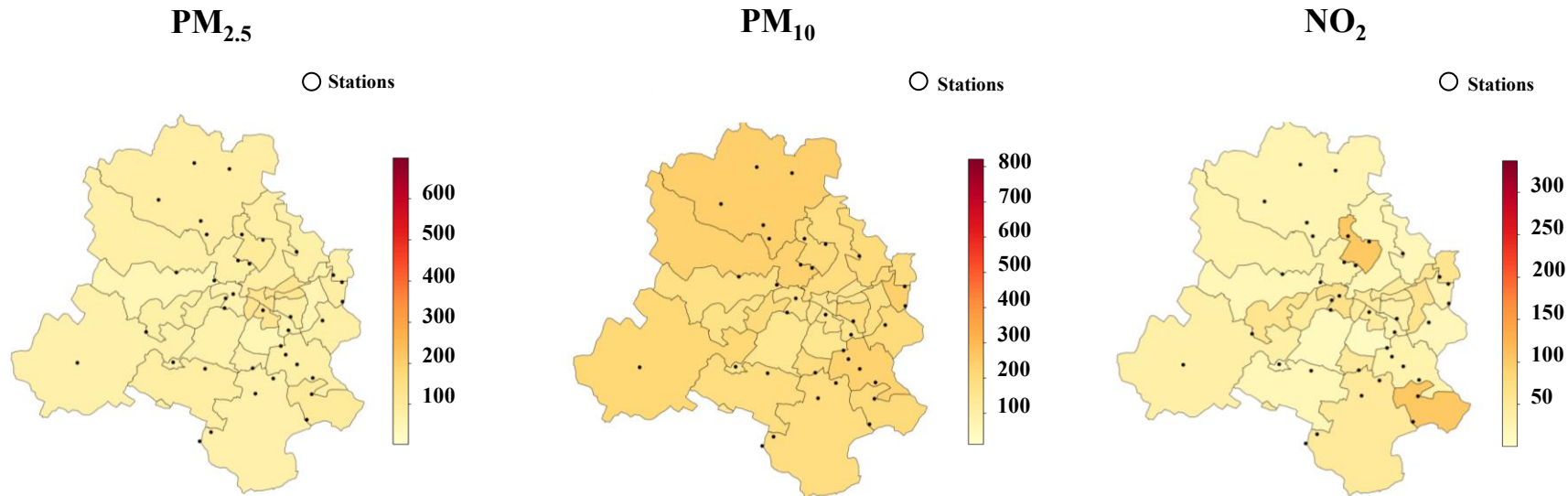


Fig: Evolution of Pollution Concentration recorded in Delhi during 2019 – 2022*.

- ✓ Nonlinear
- ✓ Non-stationary
- ✓ Long-term dependency
- ✓ Seasonality
- ✓ Extreme events

*NAAQS Thresholds for $\text{PM}_{2.5}$ is $60 \mu\text{g}/\text{m}^3$, PM_{10} is $100 \mu\text{g}/\text{m}^3$, and NO_2 is $80 \mu\text{g}/\text{m}^3$ ($\mu\text{g}/\text{m}^3$ - micrograms per cubic meter air).

Extreme Value Theory

- In classical statistics: focus on average behavior of stochastic process (Central Limit Theorem)
- In extreme value theory: focus on extreme and rare events (Fisher-Tippett Theorem)
- Block Maxima Method

$$M_n = \max\{X_1, \dots, X_n\}$$

M_n follows a Generalized Extreme Value distribution

- Peaks over Threshold (PoT)

$$\{X_i - \tau \mid X_i > \tau\}$$

very large threshold τ follow a Generalized Pareto distribution (GPD)

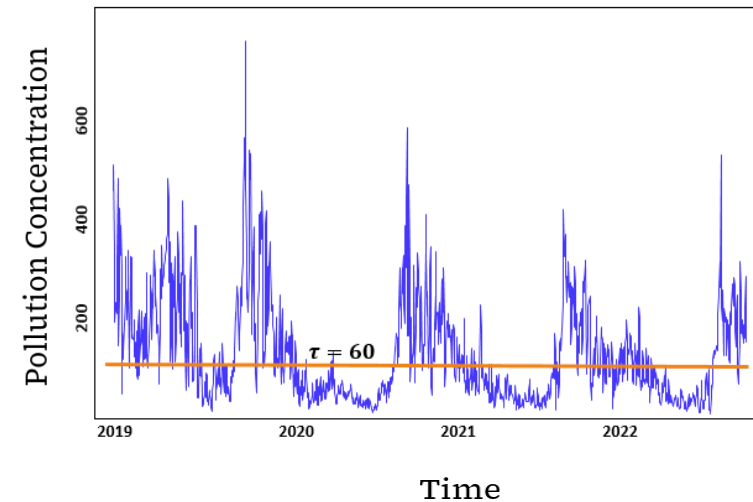


Fig: PoT method on PM_{2.5} level in Alipur, Delhi

Extreme Values in AQI Data

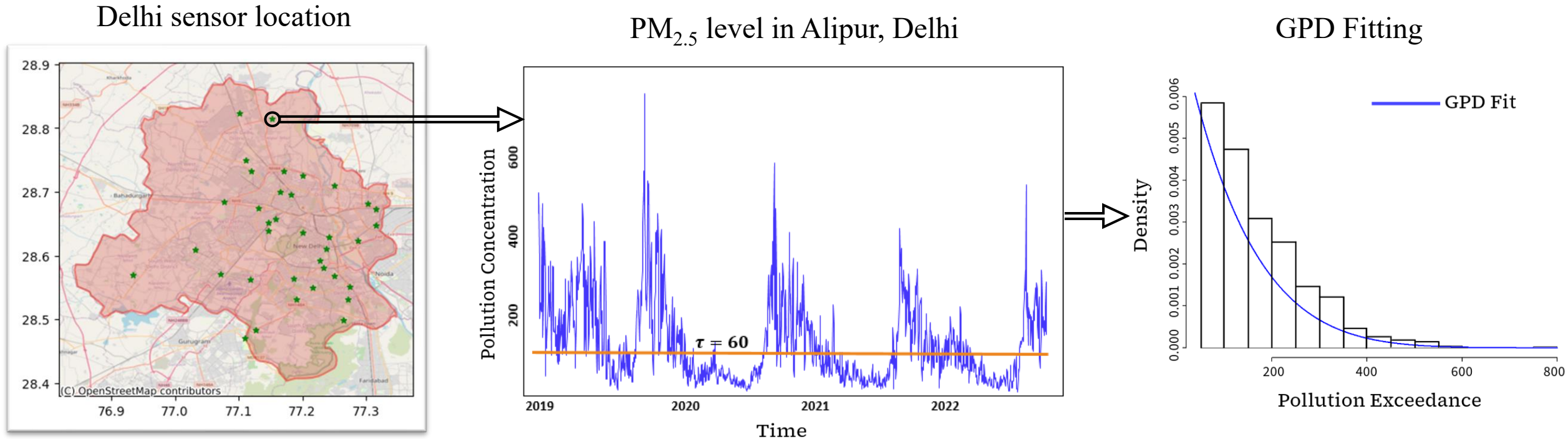


Fig: PoT analysis of PM_{2.5} concentration level recorded daily during 2019 – 2022 at Alipur, Delhi

- In Delhi, the air pollution level increases with the onset of winter due to the land-locked geographical location and low temperature restricting the dispersion of the pollutants (Extreme measurements arise).
- How can we accurately forecast AQIs for Delhi in the presence of extremes?

E-STGCN Framework

- **Input:** Pollutant concentrations across N sensors at timestamp t , $\mathbf{X}_t = \{X_t^1, \dots, X_t^N\} \in \mathbb{R}^N$; $t = 1, \dots, T$.
- **Output:** q (≥ 1) steps ahead forecast $\hat{\mathbf{X}}_{T+1}, \dots, \hat{\mathbf{X}}_{T+q}$ based on T historical observations.
- **Procedure:**

- Design a spatiotemporal graph with nodes (sensor locations) and edges (interconnections).
- The input can be represented as $\mathcal{G} = \{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_T\}$ with $\mathcal{G}_t = \{\mathbf{X}_t, A\}$ being the graph at time t and $A \in \mathbb{R}^{N \times N}$ being the static adjacency matrix.
- Compute $A = [a_{ij}]_{N \times N}$ using the **weighted Haversine distance** as

$$d_{ij} = 2R \sin^{-1} \left[\sqrt{\sin^2 \left(\frac{\phi^i - \phi^j}{2} \right) + \cos(\phi^i) \cos(\phi^j) \sin^2 \left(\frac{\lambda^i - \lambda^j}{2} \right)} \right]$$

$$a_{ij} = \exp \left(-\frac{d_{ij}^2}{\sigma^2} \right), \text{ when } i \neq j, \exp \left(-\frac{d_{ij}^2}{\sigma^2} \right) \geq \epsilon$$

where ϕ^i, λ^i is the latitude, longitude of sensor i , with σ^2, ϵ controlling the sparsity of A , and R represents Earth's radius.

E-STGCN: Spatial Module

- Encode the structural and feature-based aspects of the graph using GCNs (K layers) through neural message passing mechanism for node i as follows:

$$h_t^{i,(0)} = X_t^i$$
$$h_t^{i,(k)} = f_t^{(k)} \left(W_t^{(k)} \frac{\sum_{j \in \mathcal{N}(i)} h_t^{j,(k-1)}}{|\mathcal{N}(i)|} + B_t^{(k)} h_t^{i,(k-1)} \right) ; k = 1, 2, \dots, K$$

- Generate the spatiotemporal embedding based on the GCNs' representations using an MLP as:

$$Z_t^i = \text{Dense} \left(h_t^{i,(K)} \right)$$

E-STGCN: Temporal Module

- Provide the spatiotemporal embedding $\mathbf{Z}_t = \{Z_t^1, \dots, Z_t^N\}$ to the **Temporal Block: LSTM layers** and a **Dense layer** to generate the desired forecasts.
- The mathematical formulation of the chain-like architecture in LSTM with input gate ($\mathbf{I}_t$), forget gate ($\mathbf{F}_t$), and output gate ($\mathbf{O}_t$) can be expressed as:

$$\begin{aligned}\mathbf{F}_t &= \text{ReLU}(U^F \mathbf{Y}_t + B^F); \mathbf{I}_t = \text{ReLU}(U^I \mathbf{Y}_t + B^I); \\ \mathbf{M}_t &= \tanh(U^M \mathbf{Y}_t + B^M); \mathbf{C}_t = (\mathbf{F}_t \odot \mathbf{C}_{t-1} \oplus \mathbf{I}_t \odot \mathbf{M}_t); \\ \mathbf{O}_t &= \text{ReLU}(U^O \mathbf{Y}_t + B^O); \mathbf{H}_t = (\mathbf{O}_t \odot \tanh(\mathbf{C}_t)),\end{aligned}$$

where $\mathbf{Y}_t$ is the concatenation matrix with p lagged values of $\mathbf{Z}_t$ and previous hidden state $\mathbf{H}_{t-1}$, the new memory $\mathbf{M}_t$ at time t is used to update the cell state $\mathbf{C}_t$ (long term memory) and hidden state $\mathbf{H}_t$ (short term memory).

- Based on the computed values of $\mathbf{C}_t$ and $\mathbf{H}_t$ the q -steps-ahead forecast of $\mathbf{Z}_t$ is computed as:

$$\{\hat{\mathbf{Z}}_{t+1}, \hat{\mathbf{Z}}_{t+2}, \dots, \hat{\mathbf{Z}}_{t+q}\} = \text{ReLU}(U^H \mathbf{H}_t + B^H)$$

where U and B represents the recurrent weights matrix and the bias vector to be optimized based on the **proposed loss function**.

E-STGCN: EVT Module

- EVT module performs PoT approach on the pollutant data.
- Hence, the conditional GPD for the i^{th} monitoring station at time t is:

$$P[X_t^i - \tau \leq x_t^i \mid X_t^i \geq \tau] = \begin{cases} 1 - \left(1 + \frac{\xi^i x_t^i}{\alpha^i}\right)^{-1/\xi_t^i}, & \xi^i \neq 0 \\ 1 - \exp\left(-\frac{x_t^i}{\alpha^i}\right)^2, & \xi^i = 0 \end{cases}$$

where $\xi^i (\in \mathbb{R})$ is the shape parameter and $\alpha^i (> 0)$ is the scale parameter.

- The POTL loss function is the deviance or **negative log-likelihood** loss from GPD

$$PoTL(z_t^i) = \text{NLL}(\text{GPD}) = \log g(\hat{\alpha}^i) + \left(1 + \frac{1}{\hat{\xi}^i}\right) \log g\left(1 + \frac{\hat{\xi}^i z_t^i}{\hat{\alpha}^i}\right)$$

for location i for one data point. **Lower NLL = Better Model Fit**

E-STGCN: Optimization

- Optimize the network using the following loss function (based on the [data loss](#) and the [PoT-based loss](#)):

$$\text{Loss}(Z_{t+q}^i, \hat{Z}_{t+q}^i) = \begin{cases} \text{MSE}(Z_{t+q}^i, \hat{Z}_{t+q}^i), & \hat{Z}_{t+q}^i \leq \tau, \\ \boxed{\beta_1} \text{MSE}(Z_{t+q}^i, \hat{Z}_{t+q}^i) + \boxed{\beta_2} \text{POTL}(\hat{Z}_{t+q}^i), & \hat{Z}_{t+q}^i > \tau, \end{cases}$$

Hyperparameters
(cross validation)

- Following PINN, the final loss function of the LSTM network is formulated as a combination of the data loss and the [POT-based loss function whenever the predicted values exceed the NAAQS threshold \$\tau\$](#) .
- Since the loss function is differentiable almost everywhere, we employ the backpropagation approach to train the corresponding weights.
- The modified loss function employed for handling the temporal dependency in the E-STGCN framework integrates prior distributional knowledge to the predictions in case of threshold exceedances.

Statistical Significance

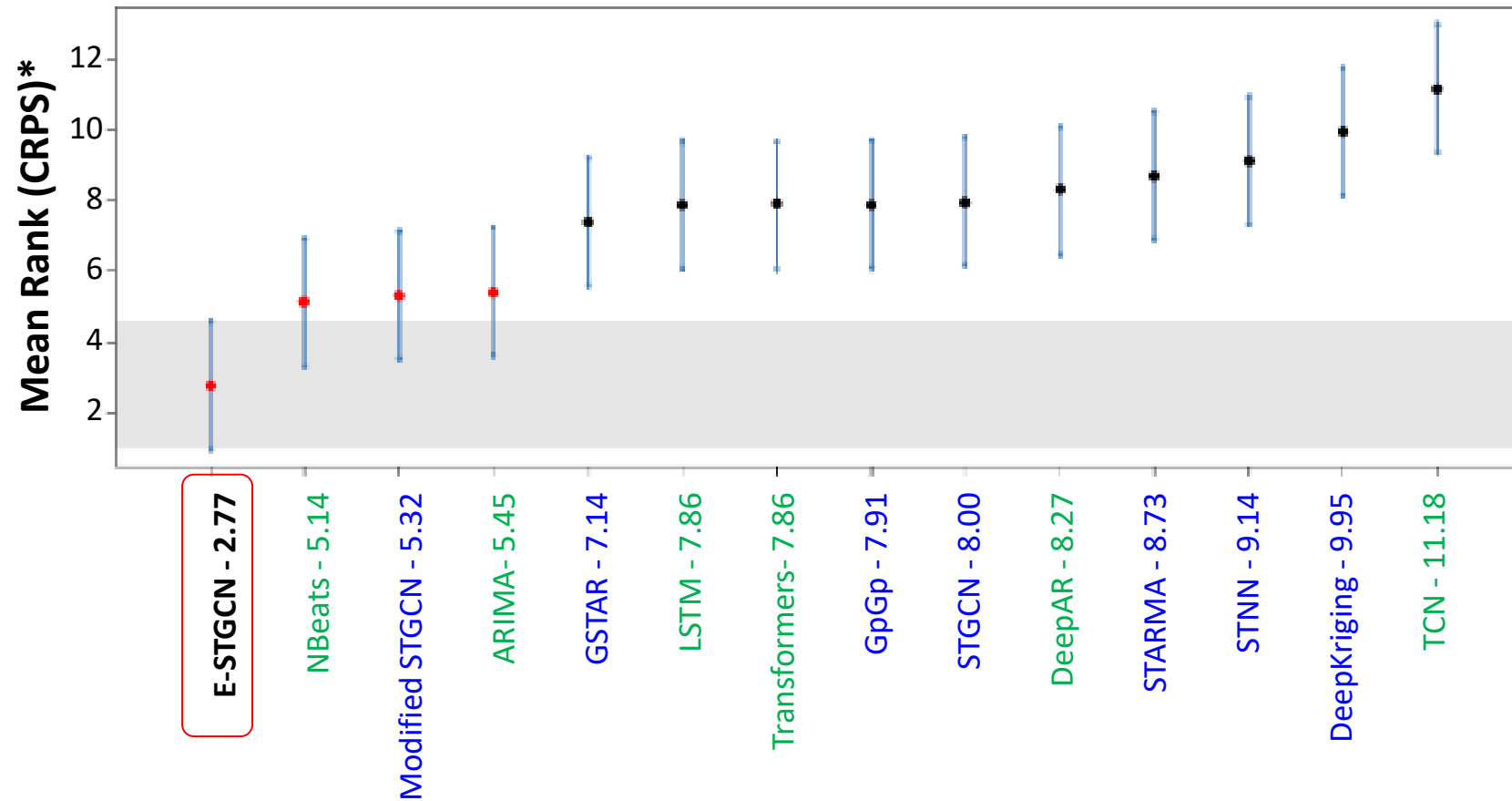


Fig: MCB test comparing the performance of E-STGCN with **spatiotemporal** and **temporal** models for PM_{2.5} forecasting based on CRPS metric

*Continuous Ranked Probability Score (CRPS): Gneiting, T., & Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. Journal of the American Statistical Association, 102(477), 359-378.

Uncertainty Quantification

- Going **beyond point forecast** by providing valid prediction intervals for individual forecasts.
- E-STGCN leverages the distribution-free **Conformal Prediction*** (CP) technique for estimating uncertainty in predictions.
- In CP method, **conformal score** (S_t) is calculated based on the absolute residuals standardized with the conditional absolute deviation (uncertainty model: $\hat{Y}_t^U$)

$$S_t = \frac{|Y_t - \hat{Y}_t|}{\hat{Y}_t^U}$$

- The CP interval for prediction $\hat{Y}_t$ at time step t can be computed using the **conformal quantiles** ($\hat{\mathbb{Q}}_t$) as:

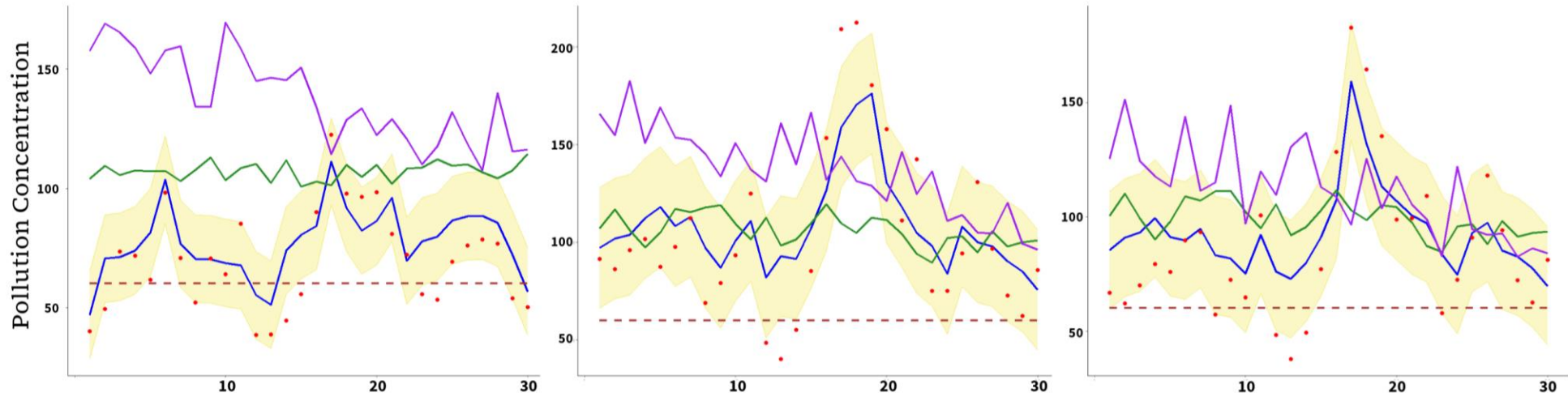
$$\mathbb{C}(\hat{Y}_t) = [\hat{Y}_t - \hat{\mathbb{Q}}_t \hat{Y}_t^U, \hat{Y}_t + \hat{\mathbb{Q}}_t \hat{Y}_t^U]$$

*Xu, Chen, and Yao Xie. "Conformal prediction interval for dynamic time-series." ICML 2021.

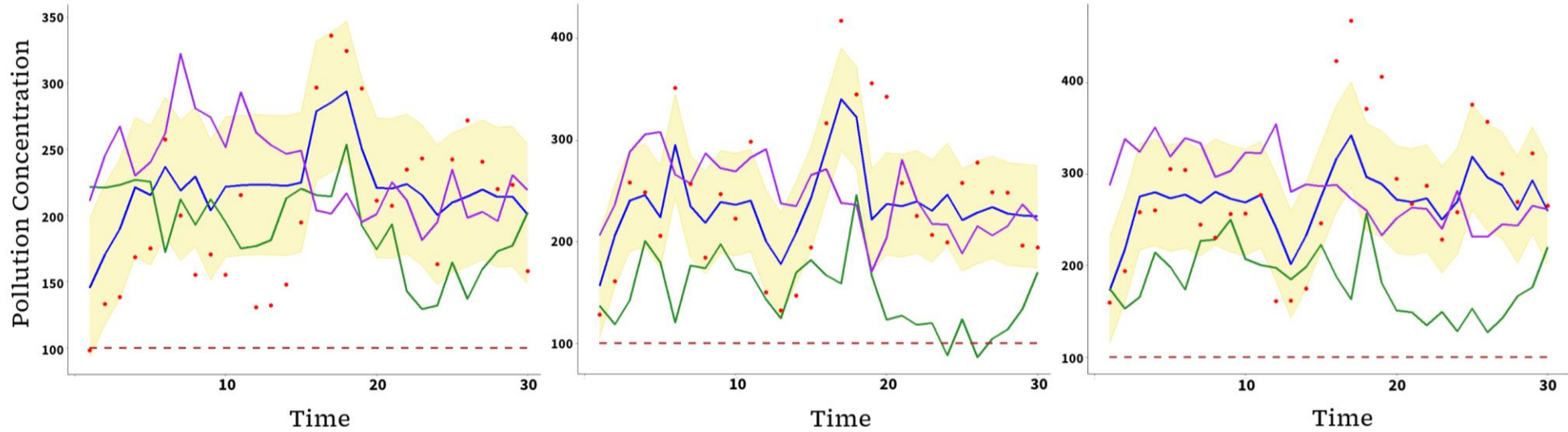
E-STGCN Prediction (Feb 2023)



$PM_{2.5}$



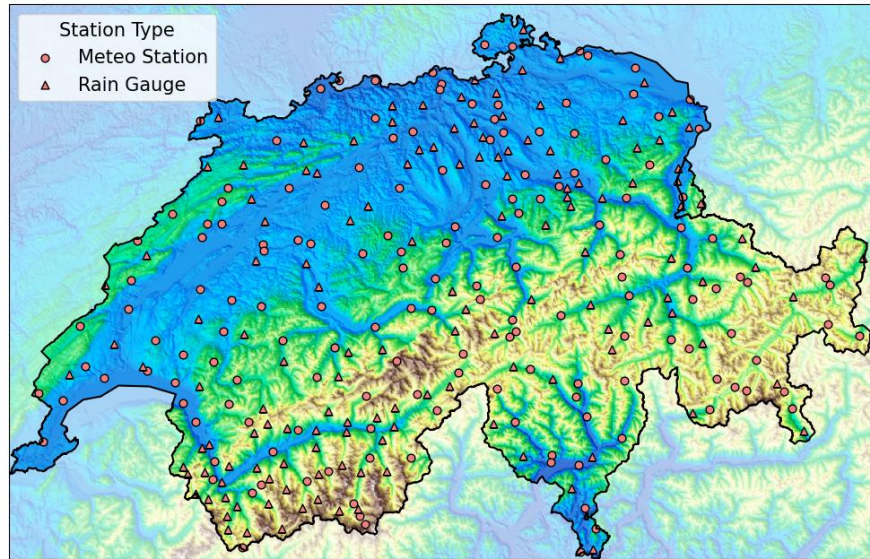
PM_{10}



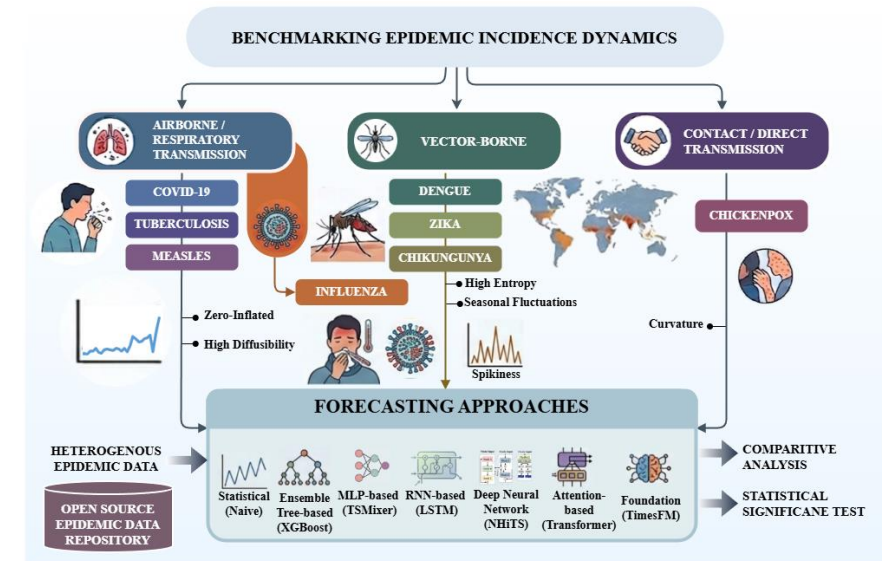


Open Data and Code

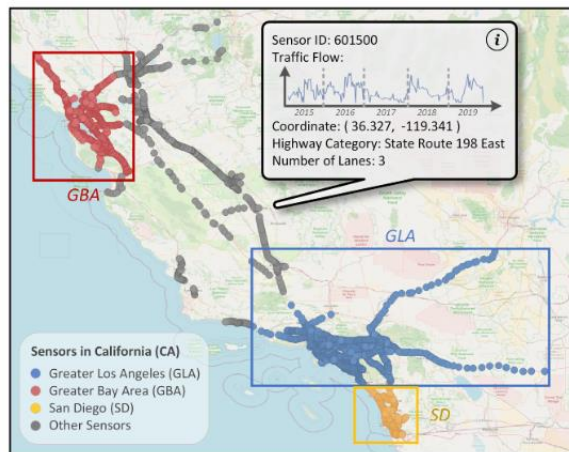
Benchmark Open ST Datasets



PeakWeather Dataset: <https://peakweather.readthedocs.io/latest/>



EpiCastBench: <https://github.com/aimltsf/EpiCastBench>



LargeST: <https://github.com/liuxu77/LargeST>



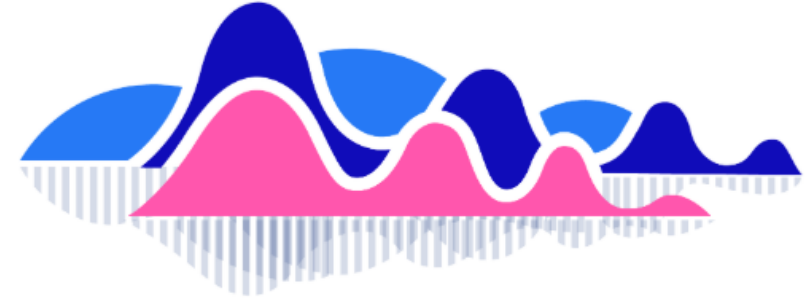
INDIA's AQI data: <https://airquality.cpcb.gov.in/ccr/>

Open ST Forecasting Software



PyG

Link: https://github.com/pyg-team/pytorch_geometric



Basic TS

Link: <https://github.com/GestaltCogTeam/BasicTS>



Link: <https://torch-spatiotemporal.readthedocs.io/>

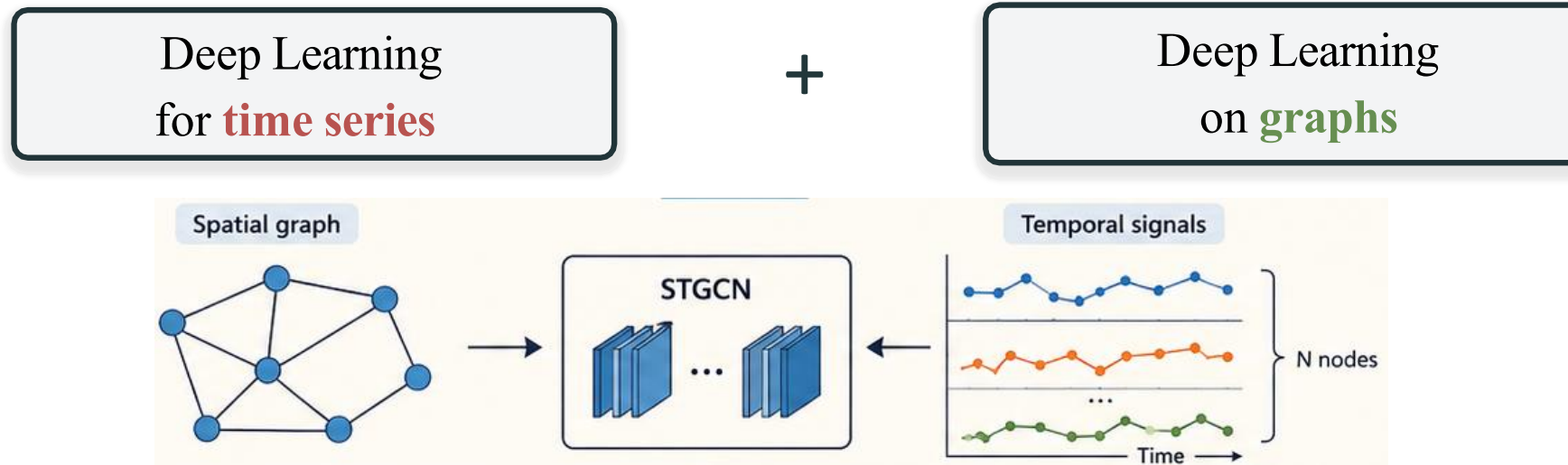


Link: <https://stengression.readthedocs.io/>



Conclusion and Future Direction

Conclusion



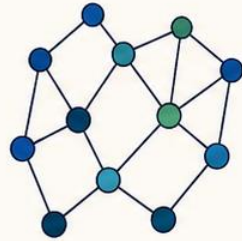
- **Deep learning on graphs works and is very effective!**
- Exciting area: lots of new applications and extensions (hard to keep up)
- **Relational** inductive **biases** allow for exploiting dependencies among the time series, ...while **sharing** most of the model **parameters** ...and overcoming limits due to **irregularities in time and space**.

Future Direction



THEORY OF STGNN

1 Generalization & Expressivity



Theorem (Informal)
For graphs $G \sim \mathcal{D}$,
 $\mathcal{L}(f_\theta(G), y) \leq \epsilon$
with high probability.

f_θ : GNN model
 $\mathcal{D}$: data distribution

Capacity to represent complex graph and spatiotemporal functions.

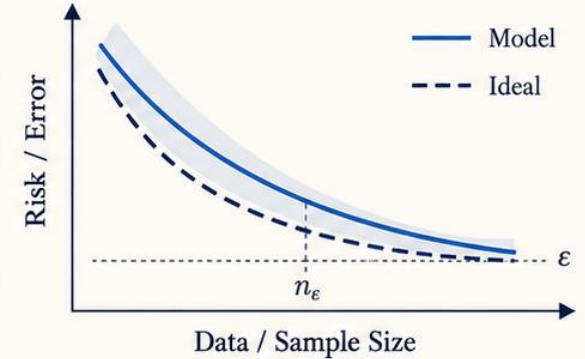
2 Stability & Identifiability



Stability (Robustness)
Small perturbations in graphs, features, or time lead to small changes in predictions.

Identifiability
Model parameters / representations are recoverable (up to symmetries).

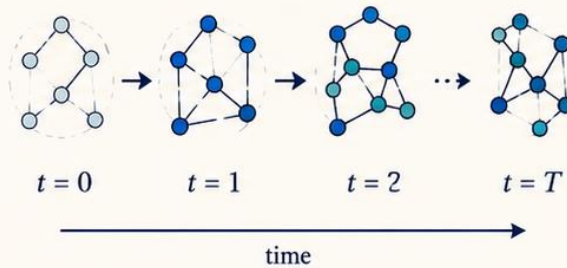
3 Learning Guarantees



Risk approaches the ideal rate with sufficient data.

SCALABLE, STABLE GENERATIVE MODELS

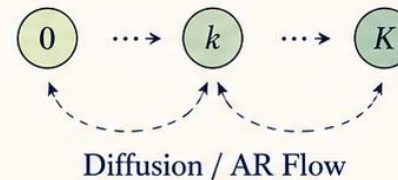
1 Graph generation over time



● generation on graphs over time

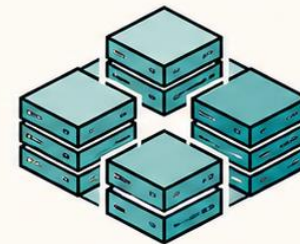
2 Generative Process

Diffusion / AR flow



● scalable training

3 Scalable Architectures



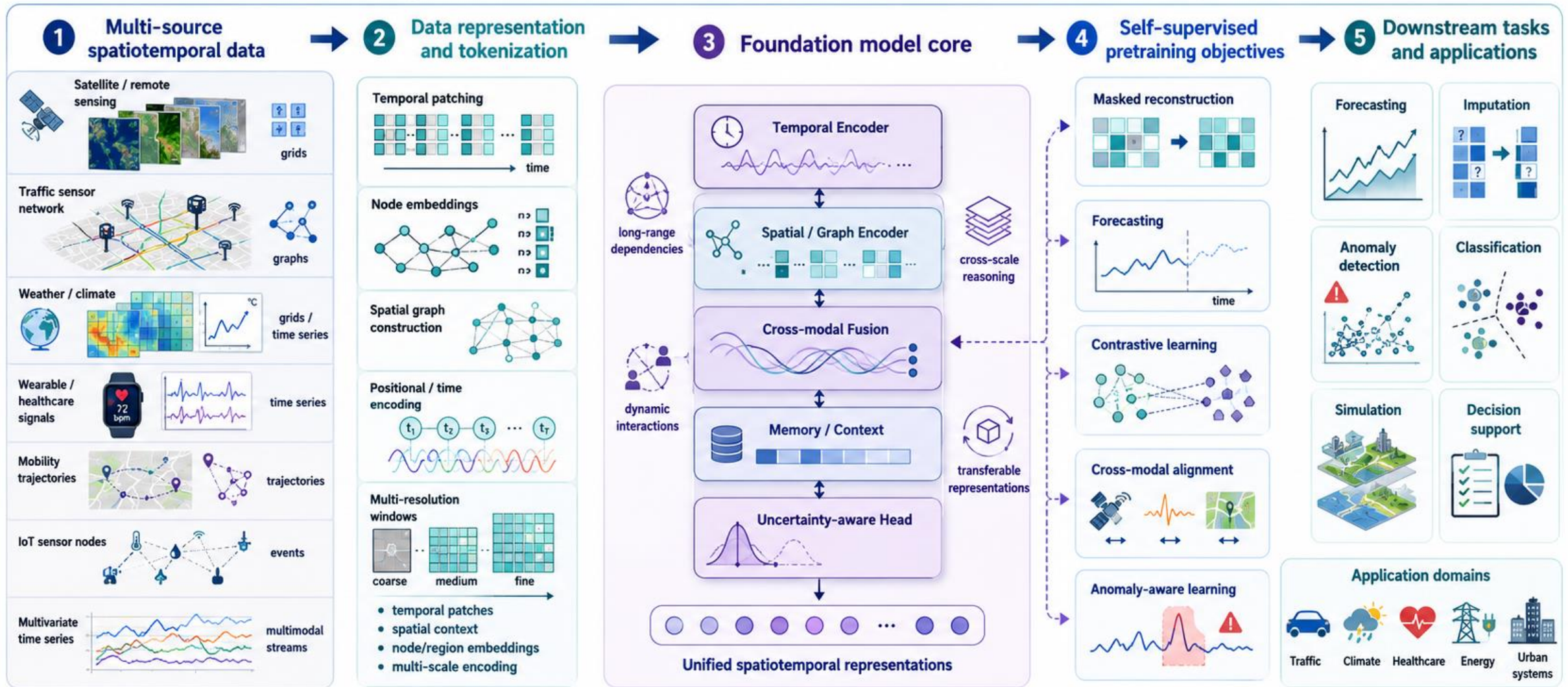
● scalable training

4 Stable & Safe Training



● robust and stable synthesis

Building a Foundation Model for Spatiotemporal Data



ForeML Lab @Sorbonne



From left to right: Mr. [Rajdeep Pathak](#), Ms. [Donia Besher](#), Dr. [Madhurima Panja](#), Dr. [Grace Younes](#), and Dr. [Tanujit Chakraborty](#)

Webpage: <https://www.ctanujit.org/members.html>

**T
H
A
N
K

Y
O
U
!**



essentially,
all models are wrong,
but some are useful

George E. P. Box

Scan for
Slides:

